claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-af2e3f7269e288069.jsonl`
- SHA-256: `88c83c1019b8f01b39b1040b5d6998b59f804bc8e56a82f913ff255cdde265ac`
- Source modified: `2026-07-08T06:16:29+00:00`
- Imported at: `2026-07-08T15:59:28+00:00`
- project: `wf_5f13fc79-bad`
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`

Transcript

1. user (2026-07-08T06:04:07.730Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to
an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).
The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):
- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical
  to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the
  SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config
  + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed
  once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing,
  never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a
wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT
report style, naming, or speculative concerns. Cite exact file:line from undefined.

Your review dimension: correctness.
Review the #521 diff for CORRECTNESS bugs in the dispatch / placement / worker-compile / spec round-trip logic. Trace: _execute_compile ? _dispatch_compile_remote ? CloudRunCompute.run_compile ? worker cmd_compile ? _stitch_reel, and _maybe_dispatch_remote's

placement/skip_validation changes. Look for: wrong data threaded through the spec, resolve/time-pair mismatches, rc/marker misreads, download_url divergence between local and cloud paths, the local-fallback-when-no-cloud path.

2. assistant (2026-07-08T06:04:13.148Z)

I'll start by locating the PR #525 / issue #521 changes in git history, since the diff paths weren't resolved.

3. user (2026-07-08T06:04:15.160Z)

```
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
233d0eb docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
da717e7 harden(validation-cache): degrade to re-validation on any cache read/replay fault
4008742 feat(validation): cache validation verdict by video_id to skip re-validation on reupload
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
5df8ff2 docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
ab03af1 fix(cloud-run): correct proto-plus Timestamp handling in execution_terminal_state
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
9a60ca9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
394d79c fix(serving): fail fast when compute=local forces weightless in-process detection
145854e fix(cloud-run): fail fast when a worker execution terminates without a done-marker
20862d7 perf(validators): decode scene_cut/blur/relevance frames in one shared pass (#512)
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
d53851b perf(validators): decode scene_cut/blur/relevance frames in one shared pass
```

4. user (2026-07-08T06:04:16.335Z)

```
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
233d0eb docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9)
---525---
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
```

5. assistant (2026-07-08T06:04:21.762Z)

Now let me get the actual diff for PR #525. Let me look at the merge commit and its diff.

6. user (2026-07-08T06:04:23.444Z)

```
commit dac6736cb23a681135b707d768014820b239a981
Author: Avi <avi.dullu@gmail.com>
Date:   Wed Jul 8 11:29:33 2026 +0530

    perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine placement
(#521) (#525)
```

```
    The compile/stitch phase ran in-process on the CPU-only 2-vCPU control-plane; the
    2026-07-08 live CUJ (rev rally-control-plane-00019, §9 of the validation-latency
    runlog) measured ~12m55s for an 11-clip / ~80s reel from a 1.83 GiB 4K source
    (~45-90s per ~7s clip ? the per-clip branding watermark forces a full 4K re-encode
    with no NVENC). Validation (183s) also runs there; both serialize on
    _LOCAL_COMPUTE_LANE, so one run pins the control-plane ~16 min ? won't scale past a
    handful of concurrent users.

    - Config-driven placement: deployment.phase_placement {validation|segment|compile}
      ? control_plane | cloud_run_l4, resolved by backend/config/placement.py. Unset ?
      today's placement, byte-identical. A config-load guard rejects the "nobody
      validates" footgun (validation on L4 without a cloud segment).
    - compile ? L4: a kind='compile' Cloud Run dispatch mirroring _maybe_dispatch_remote.
      The control-plane resolves the reel's segments, stages a compile-spec (resolved
      time-pairs + stitching config + source uri) to the bucket, dispatches
      `worker compile` on the L4, bucket-polls a done-marker, and returns the same
      {status, filepath, download_url} shape the SPA expects. The worker stitches via the
      SHARED _stitch_reel core (no forked pipeline) and needs no control-plane DB.
    - NVENC: StitchingConfig.encoder + deployment.cloud_stitch_encoder (default
      h264_nvenc) forwarded to the L4; VideoCompiler probes it once and falls back to
      libx264 (CPU) everywhere else. Lands the D16(b)-deferred NVENC-behind-a-config-knob
      now that the CUJ supplies the cost data; libx264 stays the parity-safe default.
    - validation delegation: phase_placement.validation moves the validator suite off the
      control-plane via the existing --skip-validation lever (#513), guarded so an
      in-process run always validates.
    - cloud-serving preset places compile on the L4.

    Docs: docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md (+ README pointer).
    Tests: new test_phase_placement.py, test_compile_dispatch.py; additions to
    test_compute_backend.py (run_compile), test_worker.py (cmd_compile),
    test_compiler.py (encoder selection), test_api_cloud_dispatch.py (validation
    delegation). Full suite green (1875 passed), ruff/mypy clean, coverage 85.7%.

    Co-authored-by: Claude Opus 4.8 <noreply@anthropic.com>

 backend/compute/__init__.py                        |   2 +
 backend/compute/base.py                            |  34 +++
 backend/compute/cloud_run.py                       | 100 ++++++-
 backend/config/__init__.py                         |  16 ++
 backend/config/models.py                           |  52 ++++
 backend/config/placement.py                        |  59 ++++
 backend/config/presets.py                          |  10 +-
 backend/orchestration.py                           | 322 ++++++++++++++++++---
 backend/pipeline/stitcher/compiler.py              |  94 ++++++-
 backend/worker/__main__.py                         | 139 +++++++++
 docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md  | 169 ++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/README.md           |   1 +
 tests/test_api_cloud_dispatch.py                   |  70 +++++
 tests/test_compile_dispatch.py                     | 204 +++++++++++++
 tests/test_compiler.py                             |  64 +++++
 tests/test_compute_backend.py                      | 111 ++++++++
 tests/test_config_deployment.py                    |   5 +
 tests/test_phase_placement.py                      | 138 +++++++++
 tests/test_worker.py                               | 175 ++++++++++++
```

7. user (2026-07-08T06:04:24.042Z)

bf97700facc2c7762a4cb2c7c039e7673eebab8b

8. assistant (2026-07-08T06:04:26.908Z)

Let me get the full diff and read the key changed files at their post-change state.

9. user (2026-07-08T06:04:31.248Z)

1143 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/pr525_backend.diff

10. user (2026-07-08T06:04:34.177Z)

```
diff --git a/backend/compute/__init__.py b/backend/compute/__init__.py
index 482f3c9..7f8663f 100644
--- a/backend/compute/__init__.py
+++ b/backend/compute/__init__.py
@@ -14,6 +14,7 @@ import os
 from typing import Any, Optional

 from backend.compute.base import (
+    CompileJobSpec,
     ComputeBackend,
     ComputeJobSpec,
     ComputeResult,
@@ -24,6 +25,7 @@ from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPol
 __all__ = [
     "ComputeBackend",
     "ComputeJobSpec",
+    "CompileJobSpec",
     "ComputeResult",
     "RemoteExecutionFailed",
     "get_compute_backend",
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 3ad76fc..f7afb40 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -51,6 +51,24 @@ class ComputeJobSpec:
     skip_validation: bool = False


+@dataclass(frozen=True)
+class CompileJobSpec:
+    """One reel-COMPILE job dispatched to the worker (#521). The heavy input ? the resolved
+    ``(start, end)`` time-pairs plus the control-plane's stitching/watermark config and the source
+    URI ? is staged to ``spec_uri`` (a small JSON in the bucket) rather than crammed onto argv, so
+    the worker needs no control-plane DB: it reads the spec, fetches the source, stitches on the L4
+    (NVENC), uploads the reel to ``<out_uri>/highlights/<video_id>/<output_name>``, and writes a
+    done-marker for the dispatcher's bucket-poll."""
+
+    out_uri: str  # output ROOT (gs://? bucket); highlights/<video_id>/<name> is written under it
+    spec_uri: str  # staged compile-spec JSON (time-pairs + stitching + source uri + video_id)
+    video_id: str  # the reel's video_id (deterministic reel path + marker)
+    output_name: str  # traversal-safe reel filename
+    # Raw worker ``--set k=v`` overrides (storage.output_root/backend + stitching.encoder). A tuple
+    # (not list/dict) so the spec stays frozen/hashable.
+    config_overrides: tuple[str, ...] = ()
+
+
 @dataclass(frozen=True)
 class ComputeResult:
     """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit code
@@ -85,3 +103,19 @@ class ComputeBackend(ABC):
```

```
51              ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend
is
52              still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
53              long GPU run doesn't look frozen. Implementations without a wait loop may
ignore it."""
54      +
55      +    def run_compile(
56      +        self,
57      +        spec: "CompileJobSpec",
58      +        *,
59      +        on_wait: "Callable[[float], None] | None" = None,
60      +    ) -> ComputeResult:
61      +        """Run a reel-COMPILE ``spec`` to completion and return its ``ComputeResult``
(#521).
62      +
63      +        Non-abstract with a NotImplementedError default: only backends that can offload
the ffmpeg
64      +        stitch (``CloudRunCompute``) override it ? an in-process/other backend never
dispatches a
65      +        compile. The returned ``ComputeResult.returncode`` is the worker's rc (0 ok /
non-zero =
66      +        stitch failure) and ``report`` is the worker's done-marker (carries the reel's
download_url)."""
67      +        raise NotImplementedError(
68      +            f"{type(self).__name__} does not support remote compile dispatch"
69      +        )
70      diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
71      index a4a0743..8df4246 100644
72      --- a/backend/compute/cloud_run.py
73      +++ b/backend/compute/cloud_run.py
74      @@ -37,6 +37,7 @@ from abc import ABC, abstractmethod
75       from typing import Any, Callable, List, Optional
76
77       from backend.compute.base import (
78      +    CompileJobSpec,
79           ComputeBackend,
80           ComputeJobSpec,
81           ComputeResult,
82      @@ -278,21 +279,7 @@ class CloudRunCompute(ComputeBackend):
83                   done_uri,
84               )
85
86      -        try:
87      -            marker = poll_until(
88      -                lambda: self._poll_check(done_uri, str(execution or "")),
89      -                self._policy,
90      -                label="cloud-run-infer",
91      -                logger=LOG,
92      -                # Live heartbeat (#482): each still-waiting tick reaches the caller so
93      -                # job.progress moves during the whole GPU run instead of freezing.
94      -                on_tick=on_wait,
95      -            )
96      -        except PolicyTimeout as timeout_exc:
97      -            # RemoteExecutionFailed (the fast negative path in _poll_check) is NOT a
PolicyTimeout,
98      -            # so it propagates past here uncaught ? the dispatch handler maps it to a
distinct,
99      -            # clearly-labelled failure instead of the ~65-min opaque timeout this
branch handles.
100     -            marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
101     +        marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
infer")
102              video_id = str(marker.get("video_id", ""))
103              # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
```

```
unreadable /
104              # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
105     @@ -317,6 +304,89 @@ class CloudRunCompute(ComputeBackend):
106                  execution=str(execution or ""),
107              )
108
109     +    def run_compile(
110     +        self,
111     +        spec: "CompileJobSpec",
112     +        *,
113     +        on_wait: "Callable[[float], None] | None" = None,
114     +    ) -> ComputeResult:
115     +        """Dispatch a reel-COMPILE (#521): run ``python -m backend.worker compile`` on
the L4 (NVENC),
116     +        bucket-poll the done-marker, and return the worker's rc + marker. Reuses the
EXACT
117     +        dispatch?poll?marker machinery as ``run_infer`` (``_await_marker`` ? crash-fast
+ timeout
118     +        rescue), so a stitch that crashes or overruns fails the same way a detection
does ? not a
119     +        ~65-min opaque hang."""
120     +        out_root = spec.out_uri.rstrip("/")
121     +        token = self._token or uuid.uuid4().hex
122     +        # Distinct marker prefix from infer's _dispatch/ so a compile + a detection for
the same
123     +        # bucket can never collide on a marker.
124     +        done_uri = f"{out_root}/_compile/{token}.done"
125     +        argv: List[str] = [
126     +            "compile",
127     +            "--out-uri",
128     +            spec.out_uri,
129     +            "--spec-uri",
130     +            spec.spec_uri,
131     +            "--done-uri",
132     +            done_uri,
133     +        ]
134     +        # Compile carries NO container-inline overrides: cmd_compile never consults
compute_target and
135     +        # never re-dispatches, so the only overrides are the CP-forwarded
storage/encoder knobs.
136     +        for kv in spec.config_overrides:
137     +            argv += ["--set", kv]
138     +
139     +        execution = self._client.run_job(
140     +            self._job_name, argv, region=self._region, project=self._project
141     +        )
142     +        LOG.info(
143     +            "cloud-run: dispatched COMPILE job=%s exec=%s; bucket-polling %s",
144     +            self._job_name,
145     +            execution,
146     +            done_uri,
147     +        )
148     +        marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
compile")
149     +        video_id = str(marker.get("video_id", "") or spec.video_id)
150     +        rc = int(marker.get("returncode", 1))
151     +        LOG.info(
152     +            "cloud-run: COMPILE job=%s done (video_id=%s rc=%d)",
153     +            self._job_name,
154     +            video_id,
155     +            rc,
156     +        )
157     +        # The marker carries the worker's stitch result (download_url / reel_uri /
status); surface it
```

```
158     +        # as ``report`` so the dispatcher returns the same {status, filepath,
download_url} shape the
159     +        # SPA expects. ``outputs_uri`` is the mirrored reel URI when present.
160     +        return ComputeResult(
161     +            returncode=rc,
162     +            outputs_uri=str(marker.get("reel_uri", "") or ""),
163     +            video_id=video_id,
164     +            report=marker,
165     +            execution=str(execution or ""),
166     +        )
167     +
168     +    def _await_marker(
169     +        self,
170     +        done_uri: str,
171     +        execution: object,
172     +        on_wait: "Callable[[float], None] | None",
173     +        *,
174     +        label: str,
175     +    ) -> dict:
176     +        """Bucket-poll ``done_uri`` to a completion marker ? shared by ``run_infer``
and
177     +        ``run_compile``. ``on_wait`` is the #482 live heartbeat (each still-waiting
tick reaches the
178     +        caller so job.progress moves). A ``RemoteExecutionFailed`` (the fast negative
path in
179     +        ``_poll_check``) is NOT a ``PolicyTimeout`` so it propagates uncaught ? the
dispatch handler
180     +        maps it to a distinct failure; a genuine poll TIMEOUT is handled (rescue-or-
cancel)."""
181     +        try:
182     +            return poll_until(
183     +                lambda: self._poll_check(done_uri, str(execution or "")),
184     +                self._policy,
185     +                label=label,
186     +                logger=LOG,
187     +                on_tick=on_wait,
188     +            )
189     +        except PolicyTimeout as timeout_exc:
190     +            return self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
191     +
192          def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
193              """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
194
195     diff --git a/backend/config/__init__.py b/backend/config/__init__.py
196     index 3f722c9..14cf483 100644
197     --- a/backend/config/__init__.py
198     +++ b/backend/config/__init__.py
199     @@ -19,10 +19,19 @@ from .models import (  # re-export common ones
200          Config,
201          DeploymentConfig,
202          IndexingConfig,
203     +    PhasePlacementConfig,
204          StitchingConfig,
205          StorageConfig,
206          ValidationConfig,
207      )
208     +from .placement import (
209     +    PHASE_COMPILE,
210     +    PHASE_SEGMENT,
211     +    PHASE_VALIDATION,
212     +    PLACE_CLOUD_L4,
213     +    PLACE_CONTROL_PLANE,
214     +    resolve_phase_placement,
215     +)
```

```
 from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
 from .startup import (
     StartupCheck,
@@ -52,9 +61,16 @@ __all__ = [
     "Config",
     "DeploymentConfig",
     "IndexingConfig",
+    "PhasePlacementConfig",
     "StitchingConfig",
     "StorageConfig",
     "ValidationConfig",
+    "resolve_phase_placement",
+    "PHASE_VALIDATION",
+    "PHASE_SEGMENT",
+    "PHASE_COMPILE",
+    "PLACE_CONTROL_PLANE",
+    "PLACE_CLOUD_L4",
     "PROFILE_PRESETS",
     "suggest_profile",
     "probe_cuda_available",
diff --git a/backend/config/models.py b/backend/config/models.py
index a7c997c..ce8e98a 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -364,6 +364,14 @@ class StitchingConfig(BaseModel):
     # background-court fragments), so this keeps the reel to the eye-catching long rallies.
     # OFF by default ? the detector output is unchanged, only which rallies reach the reel.
     min_rally_duration: float = Field(default=0.0, ge=0.0)
+    # Video encoder for the per-clip trim re-encode (the branding watermark forces a re-encode; the
+    # concat stays -c copy). Default ``libx264`` (CPU, works everywhere incl. CI). ``h264_nvenc`` /
+    # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-encode that
+    # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested encoder ONCE and
+    # falls back to libx264 if this ffmpeg build can't run it, so a non-GPU box never breaks. The
+    # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for the L4 run;
+    # the control-plane's own local stitch keeps libx264.
+    encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


@@ -451,6 +459,21 @@ class StorageConfig(BaseModel):
     gdrive_api: dict[str, Any] = Field(default_factory=dict)


+class PhasePlacementConfig(BaseModel):
+    """Per-phase machine placement (#521): move validation / segment(detect) / compile(stitch)
+    between the control-plane and the L4 Cloud Run worker **by config, no code change**.
+
+    Each field is ``None`` (? the legacy default for that phase ? see
+    ``backend/config/placement.py``) or an explicit ``"control_plane"`` / ``"cloud_run_l4"``.
+    ``control_plane`` runs the phase in-process on the CPU-only control-plane; ``cloud_run_l4``
+    dispatches it to the GPU/NVENC worker Job. Unset ? byte-identical to pre-#521 behaviour.
+    Resolve via ``backend.config.resolve_phase_placement(cfg, phase)`` ? never read
```

```
these directly."""
268     +
269     +    validation: Optional[Literal["control_plane", "cloud_run_l4"]] = None
270     +    segment: Optional[Literal["control_plane", "cloud_run_l4"]] = None
271     +    compile: Optional[Literal["control_plane", "cloud_run_l4"]] = None
272     +
273     +
274      class DeploymentConfig(BaseModel):
275          """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
276          knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
277     @@ -481,6 +504,35 @@ class DeploymentConfig(BaseModel):
278          # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
279          # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
280          remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
281     +    # Per-phase machine placement (#521). Unset ? today's placement (validation +
compile on the
282     +    # control-plane, segment on the L4 iff compute_target==cloud_run). Set a phase to
relocate it by
283     +    # config alone. Resolve via ``backend.config.resolve_phase_placement`` (never read
the sub-fields
284     +    # directly ? the resolver folds in the legacy compute_target default).
285     +    phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)
286     +    # Video encoder the cloud compile-dispatch forwards onto ``stitching.encoder`` for
the L4 stitch
287     +    # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the 4K re-encode the
watermark forces;
288     +    # set ``libx264`` to run the L4 stitch on CPU. Only consulted when compile is
placed on the L4;
289     +    # the control-plane's own local stitch always uses ``stitching.encoder`` (libx264
by default).
290     +    cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
291     +
292     +    @model_validator(mode="after")
293     +    def _validation_placement_needs_cloud_segment(self) -> "DeploymentConfig":
294     +        # A clip can only be validated ON the worker if the worker also DETECTS it
there ? the worker
295     +        # is only launched for the segment(detect) phase. So validation=cloud_run_l4
with segment on
296     +        # the control-plane would leave NOBODY validating (the control-plane skips, and
no worker runs)
297     +        # ? reject that footgun at config load rather than ship a silently-unvalidated
clip.
298     +        if self.phase_placement.validation == "cloud_run_l4":
299     +            seg = self.phase_placement.segment or (
300     +                "cloud_run_l4" if self.compute_target == "cloud_run" else
"control_plane"
301     +            )
302     +            if seg != "cloud_run_l4":
303     +                raise ValueError(
304     +                    "deployment.phase_placement.validation='cloud_run_l4' requires
segment to also "
305     +                    "run on the L4 ? the worker can only validate a clip it also
detects. Set "
306     +                    "phase_placement.segment='cloud_run_l4' (or
compute_target='cloud_run'), or keep "
307     +                    "validation on the control_plane."
308     +                )
309     +        return self
310
311
312      class BillingConfig(BaseModel):
```

```
diff --git a/backend/config/placement.py b/backend/config/placement.py
new file mode 100644
index 0000000..cda3ca5
--- /dev/null
+++ b/backend/config/placement.py
@@ -0,0 +1,59 @@
+"""Phase ? machine placement resolution (#521).
+
+A *phase placement* answers one question per pipeline phase ? **where does the heavy compute for
+this phase run?** ? with a config knob (``deployment.phase_placement``) instead of code. The three
+placeable phases and their values:
+
+    validation  ?  control_plane | cloud_run_l4
+    segment     ?  control_plane | cloud_run_l4   (a.k.a. "detect")
+    compile     ?  control_plane | cloud_run_l4   (a.k.a. "stitch")
+
+``control_plane`` runs the phase IN-PROCESS on the (CPU-only) control-plane service; ``cloud_run_l4``
+dispatches it to the GPU/NVENC Cloud Run worker Job (the same L4 detection already uses). This module
+is the SINGLE source of truth so ``backend.orchestration`` (segment + compile) and the worker never
+re-derive the mapping.
+
+**Default = today's behaviour (pure / additive).** When a phase's knob is unset (``None``) the
+resolver reproduces the pre-#521 placement:
+
+    validation  ?  control_plane                                  (the authoritative gate, #512/#513)
+    segment     ?  cloud_run_l4 IFF deployment.compute_target == "cloud_run", else control_plane
+    compile     ?  control_plane                                  (the in-process ffmpeg stitch)
+
+so a config with no ``phase_placement`` block behaves byte-identically to before this module existed.
+"""
+
+from __future__ import annotations
+
+from typing import Any
+
+# Phase identities (kept as constants so callers don't stringly-type the phase names).
+PHASE_VALIDATION = "validation"
+PHASE_SEGMENT = "segment"
+PHASE_COMPILE = "compile"
+PHASES: tuple[str, ...] = (PHASE_VALIDATION, PHASE_SEGMENT, PHASE_COMPILE)
+
+# Placement targets.
+PLACE_CONTROL_PLANE = "control_plane"
+PLACE_CLOUD_L4 = "cloud_run_l4"
+PLACEMENTS: tuple[str, ...] = (PLACE_CONTROL_PLANE, PLACE_CLOUD_L4)
+
+
+def resolve_phase_placement(cfg: Any, phase: str) -> str:
+    """Where does ``phase`` run ? ``"control_plane"`` or ``"cloud_run_l4"``?
+
+    Precedence: an explicit ``deployment.phase_placement.<phase>`` wins; otherwise the legacy default
+    (mirrors the pre-#521 behaviour, see the module docstring). Never raises ? an absent
+    ``deployment`` / ``phase_placement`` degrades to the legacy default.
```

```
    +    """
    +    deployment = getattr(cfg, "deployment", None)
    +    placement = getattr(deployment, "phase_placement", None)
    +    explicit = getattr(placement, phase, None) if placement is not None else None
    +    if explicit:
    +        return str(explicit)
    +    # Legacy defaults ? reproduce today's placement exactly.
    +    if phase == PHASE_SEGMENT:
    +        compute_target = (getattr(deployment, "compute_target", "") or "").strip()
    +        return PLACE_CLOUD_L4 if compute_target == "cloud_run" else PLACE_CONTROL_PLANE
    +    # validation + compile default to the control-plane (their pre-#521 home).
    +    return PLACE_CONTROL_PLANE
    diff --git a/backend/config/presets.py b/backend/config/presets.py
    index 77b20e8..0ea0c88 100644
    --- a/backend/config/presets.py
    +++ b/backend/config/presets.py
    @@ -49,7 +49,15 @@ PROFILE_PRESETS: dict[str, dict[str, Any]] = {
             "storage": {"backend": "local"},
         },
         "cloud-serving": {
    -        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
    +        "deployment": {
    +            "profile": "cloud-serving",
    +            "compute_target": "cloud_run",
    +            # #521: run the compile/stitch on the L4 worker (NVENC + 8 vCPU), not the
CPU-only
    +            # 2-vCPU control-plane where the 2026-07-08 CUJ measured ~13 min for an
11-clip 4K reel.
    +            # (validation stays on the control-plane ? the authoritative #512/#513
gate; segment is
    +            # already cloud_run_l4 via compute_target.)
    +            "phase_placement": {"compile": "cloud_run_l4"},
    +        },
             # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172
floor (D11).
             # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the
paid L4
             # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
    diff --git a/backend/orchestration.py b/backend/orchestration.py
    index 8b7851e..46ffaa4 100644
    --- a/backend/orchestration.py
    +++ b/backend/orchestration.py
    @@ -26,11 +26,20 @@ import os
     import re
     import tempfile
     import threading
    +import uuid
     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
     from urllib.parse import quote

     from backend import storage
    -from backend.config import AppConfig, StitchingConfig
    +from backend.config import (
    +    PHASE_COMPILE,
    +    PHASE_SEGMENT,
    +    PHASE_VALIDATION,
    +    PLACE_CLOUD_L4,
    +    AppConfig,
    +    StitchingConfig,
    +    resolve_phase_placement,
    +)
     from backend.pipeline.segmenters.base import segmenter_registry
     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
     from backend.pipeline.validators.base import ValidationResult, registry
    @@ -206,6 +215,24 @@ def _cloud_available(cfg) -> bool:
```

```
426          return bool(has_job and out_root)
427
428
429      +def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
430      +    """Would DETECT for this request run on the L4 worker (vs in-process)? Mirrors the
431      +    ``_maybe_dispatch_remote`` decision WITHOUT side effects (#521), so
``_execute_processing`` can
432      +    decide whether to DELEGATE validation to the worker
(``phase_placement.validation``). A per-request
433      +    picker choice wins; otherwise ``phase_placement.segment`` (whose legacy default is
cloud iff
434      +    ``compute_target==cloud_run``) gates it, and cloud must actually be available. A
pre-dispatch reject
435      +    (e.g. cloud requested but unconfigured) counts as NOT-on-cloud ? nothing runs
remotely."""
436      +    choice = (req_compute or "").strip().lower()
437      +    if choice == "local":
438      +        return False
439      +    if choice == "cloud":
440      +        return _cloud_available(cfg)
441      +    return (
442      +        resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
443      +        and _cloud_available(cfg)
444      +    )
445      +
446      +
447       def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
448           """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
449           minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
450      @@ -505,11 +532,19 @@ def _maybe_dispatch_remote(
451                   )
452               compute = get_compute_backend(cfg, target_override="cloud_run")
453           else:
454      -        compute = get_compute_backend(
455      -            cfg
456      -        )  # server default (default-OFF unless cloud_run)
457      +        # No explicit picker ? consult phase_placement.segment (#521). Its legacy
default is
458      +        # cloud_run_l4 IFF compute_target==cloud_run, so this preserves today's
behaviour; setting
459      +        # phase_placement.segment relocates DETECT by config alone (e.g. force it back
on the
460      +        # control-plane, or onto the L4 on a default-local box wired for cloud).
461      +        if (
462      +            resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
463      +            and _cloud_available(cfg)
464      +        ):
465      +            compute = get_compute_backend(cfg, target_override="cloud_run")
466      +        else:
467      +            compute = None
468           if compute is None:
469      -        return None  # default-OFF ? run the in-process segmenter below (unchanged)
470      +        return None  # default-OFF / placed on control_plane ? the in-process segmenter
below
471
472           storage_cfg = getattr(cfg, "storage", None)
473
474      @@ -623,12 +658,20 @@ def _maybe_dispatch_remote(
475           # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
476           # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
477           # then worker-rejected at its default 20.0 after a successful GPU dispatch).
478      +    # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the
```

```
control-plane is
479     +    # the authoritative gate ? it already validated with its resolved config (only a
PASS reaches
480     +    # here), so the worker SKIPS its redundant re-validation (#513). If validation is
placed on the
481     +    # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators
? see
482     +    # _execute_processing), so the worker must NOT skip: it becomes the gate.
483     +    worker_skips_validation = (
484     +        resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
485     +    )
486          spec = ComputeJobSpec(
487              video_uri=req.video_path,
488              out_uri=out_root,
489              segmenter=seg_name,
490              config_overrides=tuple(overrides),
491     -        skip_validation=True,
492     +        skip_validation=worker_skips_validation,
493          )
494          try:
495              # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
496     @@ -811,12 +854,33 @@ def _execute_processing(
497              # ``force=true`` bypasses the reuse and re-writes a fresh verdict below
(mirrors how it bypasses
498              # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
499              # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
500     -        val_fingerprint = registry.config_fingerprint(config)
501     +        #
502     +        # #521 phase_placement.validation: when validation is placed on the L4 AND
detection will
503     +        # actually dispatch there, the control-plane DELEGATES validation to the worker
? it does NOT
504     +        # run the validator suite here (the whole point is to move the CPU-bound
validation OFF the
505     +        # 2-vCPU control-plane). The worker then validates (skip_validation=False) and
rejects a bad
506     +        # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed
job. When detection
507     +        # is NOT on the cloud (in-process), the control-plane always validates ? there
is no worker to
508     +        # delegate to (the config guard makes validation=cloud_run_l4 require a cloud
segment).
509     +        validate_on_control_plane = (
510     +            resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
511     +            or not _will_detect_on_cloud(config, getattr(req, "compute", None))
512     +        )
513     +        if not validate_on_control_plane:
514     +            logger.info(
515     +                "[API] validation delegated to the L4 worker "
516     +                "(deployment.phase_placement.validation=cloud_run_l4) ? the control-
plane skips its "
517     +                "validator suite; the worker is the gate for %s.",
518     +                req.video_path,
519     +            )
520     +        val_fingerprint = (
521     +            registry.config_fingerprint(config) if validate_on_control_plane else ""
522     +        )
523              # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
524              # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
525              # failing the job. The cache is a pure speedup ? never a new failure mode.
526              reused_validation = False
```

```
527     -        if not req.force:
528     +        if validate_on_control_plane and not req.force:
529                  try:
530                      cached = db.get_validation_cache(video_id)
531                      if cached is not None and cached.get("fingerprint") == val_fingerprint:
532     @@ -841,11 +905,11 @@ def _execute_processing(
533                          video_id,
534                          exc_info=True,
535                      )
536     -        if not reused_validation:
537     +        if validate_on_control_plane and not reused_validation:
538                  logger.info(f"Running validations for {req.video_path}...")
539                  val_results, val_context = registry.run_all_with_context(local_video,
config)
540              failures = [r for r in val_results if not r.passed]
541     -        if not reused_validation:
542     +        if validate_on_control_plane and not reused_validation:
543                  # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
544                  # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
545                  db.set_validation_cache(
546     @@ -1114,28 +1178,34 @@ def signed_reel_url(cfg, video_id: str, name: str) ->
Optional[str]:
547          return None
548
549
550     -def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
551     -    """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
552     -    and return the {status, filepath, download_url} result the SPA polls for. Returns a
553     -    ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
554     -    so the SPA always gets a clean verdict instead of a worker-crash error."""
555     -    import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
556     +def _compile_wait_message(elapsed_sec: float) -> str:
557     +    """The live job.progress line while an L4 compile/stitch is in flight (#521;
mirrors
558     +    ``_wait_progress_message``). Minute-granular; under a minute reads as cold-
start."""
559     +    minutes = int(elapsed_sec) // 60
560     +    if minutes < 1:
561     +        return "stitching on cloud GPU (NVENC) ? starting up"
562     +    return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
563
564     -    notify = progress or (lambda stage: None)
565     -    params = json.loads(job.get("params") or "{}")
566     -    video_id = job.get("video_id") or ""
567     -    segment_ids = params.get("segment_ids") or []
568     -    locale = params.get("locale")
569
570     -    notify("resolving")
571     -    try:
572     -        video, kept, out_name = _resolve_compile(
573     -            db, cfg, video_id, segment_ids, params.get("output_filename", "")
574     -        )
575     -    except _CompileError as e:
576     -        return {"status": "failed", "message": e.detail, "video_id": video_id}
577     +def _stitch_reel(
578     +    cfg,
579     +    stitching,
580     +    video_id: str,
581     +    source_ref: str,
582     +    time_pairs: List[Tuple[float, float]],
```

```
583     +    out_name: str,
584     +    locale: Optional[str],
585     +    notify,
586     +) -> Dict[str, Any]:
587     +    """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
588     +    (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
589     +    needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
590     +    with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the
source, runs the
591     +    trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
592     +    returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
593     +    ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
594     +    import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
595
596     -    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
597          output_dir = (
598              getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
599          )
600     @@ -1147,8 +1217,8 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None)
-> Dict[str, A
601              begin_padding=stitching.begin_padding,
602              end_padding=stitching.end_padding,
603              watermark=localized_watermark,
604     +        encoder=getattr(stitching, "encoder", "libx264"),
605          )
606     -    time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
607
608          notify("stitching")
609          local_src = ""
610     @@ -1157,7 +1227,7 @@ def _execute_compile(cfg, db, job: Dict[str, Any], progress=None)
-> Dict[str, A
611              # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
612              # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
613              local_src = storage.acquire_local_input(
614     -            video["filepath"], getattr(cfg, "storage", None)
615     +            source_ref, getattr(cfg, "storage", None)
616              )
617              # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
618              # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
619     @@ -1175,13 +1245,14 @@ def _execute_compile(cfg, db, job: Dict[str, Any],
progress=None) -> Dict[str, A
620          finally:
621              if local_src:
622                  storage.cleanup_acquired_local_input(
623     -                video["filepath"], local_src, getattr(cfg, "storage", None)
624     +                source_ref, local_src, getattr(cfg, "storage", None)
625                  )
626
627          # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
628          # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
629          # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
```

```
    -    # still falls back to the local file on the same instance).
    +    # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
    +    # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
         reel_uri = reel_object_uri(cfg, video_id, out_name)
         if reel_uri:
             try:
    @@ -1201,4 +1272,189 @@ def _execute_compile(cfg, db, job: Dict[str, Any],
progress=None) -> Dict[str, A
             "filepath": out_path,
             "download_url": download_url,
             "video_id": video_id,
    +        "reel_uri": reel_uri or "",
         }
    +
    +
    +def _dispatch_compile_remote(
    +    cfg,
    +    db,
    +    video_id: str,
    +    segment_ids: List[int],
    +    output_filename: str,
    +    locale: Optional[str],
    +    notify,
    +) -> Dict[str, Any]:
    +    """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
    +    control-plane owns the DB, so it RESOLVES the reel's segments here
(``_resolve_compile``), stages a
    +    small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
    +    to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
    +    done-marker, and returns the same ``{status, filepath, download_url}`` shape the
SPA expects. The
    +    worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
    +    ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
    +    from backend.compute import CompileJobSpec, get_compute_backend
    +
    +    notify("resolving")
    +    try:
    +        video, kept, out_name = _resolve_compile(
    +            db, cfg, video_id, segment_ids, output_filename
    +        )
    +    except _CompileError as e:
    +        return {"status": "failed", "message": e.detail, "video_id": video_id}
    +
    +    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
    +    storage_cfg = getattr(cfg, "storage", None)
    +    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
    +    compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
    +    if compute is None:
    +        # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
    +        # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail
? a slow reel
    +        # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
    +        logger.warning(
    +            "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
    +            "stitching in-process on the control-plane instead.",
    +            out_root,
```

```
682     +        )
683     +        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
684     +        return _stitch_reel(
685     +            cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
686     +        )
687     +
688     +    # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-
safe on the wire);
689     +    # the stitching config is the control-plane's RESOLVED one, with the encoder
swapped to the L4's
690     +    # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-
encodes the reel.
691     +    stitch_dump = stitching.model_dump()
692     +    stitch_dump["encoder"] = getattr(
693     +        getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
694     +    )
695     +    spec_payload = {
696     +        "video_id": video_id,
697     +        "video_uri": video["filepath"],
698     +        "output_name": out_name,
699     +        "locale": locale,
700     +        "time_pairs": [
701     +            [float(s["start_time"]), float(s["end_time"])] for s in kept
702     +        ],
703     +        "stitching": stitch_dump,
704     +    }
705     +    token = uuid.uuid4().hex
706     +    spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
707     +    try:
708     +        notify("staging compile spec")
709     +        with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
710     +            local_spec = os.path.join(td, "spec.json")
711     +            with open(local_spec, "w", encoding="utf-8") as f:
712     +                json.dump(spec_payload, f)
713     +            storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
714     +    except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
715     +        logger.error(
716     +            "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
717     +        )
718     +        return {
719     +            "status": "failed",
720     +            "message": f"cloud compile: could not stage the compile spec: {e}",
721     +            "video_id": video_id,
722     +        }
723     +
724     +    # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
725     +    # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
726     +    overrides = [f"storage.output_root={out_root}"]
727     +    backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
728     +    if backend_name:
729     +        overrides.append(f"storage.backend={backend_name}")
730     +    spec = CompileJobSpec(
731     +        out_uri=out_root,
732     +        spec_uri=spec_uri,
733     +        video_id=video_id,
734     +        output_name=out_name,
735     +        config_overrides=tuple(overrides),
736     +    )
737     +
738     +    notify("dispatching stitch (cloud_run)")
```

```
739     +    logger.info(
740     +        "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
741     +        video_id,
742     +        out_root,
743     +        len(kept),
744     +    )
745     +    try:
746     +        result = compute.run_compile(
747     +            spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
748     +        )
749     +    except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
750     +        logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
751     +        return {
752     +            "status": "failed",
753     +            "message": f"cloud compile dispatch error: {e}",
754     +            "video_id": video_id,
755     +        }
756     +    if result.returncode != 0:
757     +        msg = str(
758     +            result.report.get("message")
759     +            or f"cloud compile job failed (returncode {result.returncode})."
760     +        )
761     +        return {"status": "failed", "message": msg, "video_id": video_id}
762     +
763     +    download_url = str(
764     +        result.report.get("download_url")
765     +        or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
766     +    )
767     +    logger.info(
768     +        "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
769     +        result.execution,
770     +        result.outputs_uri,
771     +        video_id,
772     +    )
773     +    notify("finished")
774     +    return {
775     +        "status": "success",
776     +        "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
777     +        "download_url": download_url,
778     +        "video_id": video_id,
779     +        "compute": {
780     +            "path": "cloud_run",
781     +            "execution": result.execution,
782     +            "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
783     +            "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
784     +        },
785     +    }
786     +
787     +
788     +def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
789     +    """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
790     +    the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
791     +    result for an EXPECTED problem (missing video / no segments / a stitch error) so
the SPA always gets
792     +    a clean verdict instead of a worker-crash error.
793     +
794     +    #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``)
and cloud
795     +    dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
796     +    (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
```

```
797     +    2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process
here, unchanged."""
798     +    notify = progress or (lambda stage: None)
799     +    params = json.loads(job.get("params") or "{}")
800     +    video_id = job.get("video_id") or ""
801     +    segment_ids = params.get("segment_ids") or []
802     +    locale = params.get("locale")
803     +    output_filename = params.get("output_filename", "")
804     +
805     +    if (
806     +        resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
807     +        and _cloud_available(cfg)
808     +    ):
809     +        return _dispatch_compile_remote(
810     +            cfg, db, video_id, segment_ids, output_filename, locale, notify
811     +        )
812     +
813     +    notify("resolving")
814     +    try:
815     +        video, kept, out_name = _resolve_compile(
816     +            db, cfg, video_id, segment_ids, output_filename
817     +        )
818     +    except _CompileError as e:
819     +        return {"status": "failed", "message": e.detail, "video_id": video_id}
820     +
821     +    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
822     +    time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
823     +    return _stitch_reel(
824     +        cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
825     +    )
826     diff --git a/backend/pipeline/stitcher/compiler.py
b/backend/pipeline/stitcher/compiler.py
827     index 65d1c57..5c7229e 100644
828     --- a/backend/pipeline/stitcher/compiler.py
829     +++ b/backend/pipeline/stitcher/compiler.py
830     @@ -23,6 +23,49 @@ _BUNDLED_FONT = (
831          Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
832      )
833
834     +#: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's
first clip pays
835     +#: it and the rest reuse the verdict. Keyed by encoder name; cleared only across
processes.
836     +_ENCODER_AVAILABLE: Dict[str, bool] = {}
837     +
838     +
839     +def _probe_encoder_available(encoder: str) -> bool:
840     +    """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny
1-frame
841     +    lavfi test encode to null. NVENC is often *compiled in* on boxes without an NVIDIA
GPU, so a
842     +    real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from
"will fail at
843     +    runtime". Memoized. Any error / timeout ? unavailable (caller falls back to
libx264)."""
844     +    if encoder in _ENCODER_AVAILABLE:
845     +        return _ENCODER_AVAILABLE[encoder]
846     +    ok = False
847     +    # The probe is a tiny 1-frame encode ? bound it to a short timeout (? the
configured ffmpeg
848     +    # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
849     +    configured = ffmpeg_timeout_sec()
850     +    probe_timeout = min(30.0, float(configured)) if configured else 30.0
851     +    try:
```

```
852     +        proc = subprocess.run(
853     +            [
854     +                ffmpeg_bin(),
855     +                "-hide_banner",
856     +                "-f",
857     +                "lavfi",
858     +                "-i",
859     +                "color=c=black:s=64x64:d=0.1",
860     +                "-frames:v",
861     +                "1",
862     +                "-c:v",
863     +                encoder,
864     +                "-f",
865     +                "null",
866     +                "-",
867     +            ],
868     +            capture_output=True,
869     +            timeout=probe_timeout,
870     +        )
871     +        ok = proc.returncode == 0
872     +    except Exception:  # noqa: BLE001 ? a probe failure/timeout just means "not usable
? fall back"
873     +        ok = False
874     +    _ENCODER_AVAILABLE[encoder] = ok
875     +    return ok
876     +
877
878      class VideoCompiler:
879          def __init__(
880     @@ -31,6 +74,7 @@ class VideoCompiler:
881              end_padding: float = 1.0,
882              begin_padding: float = 0.5,
883              watermark: Optional[Any] = None,
884     +        encoder: str = "libx264",
885          ):
886              self.output_dir = output_dir
887              self.end_padding = end_padding
888     @@ -38,6 +82,11 @@ class VideoCompiler:
889              # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
890              # normalized to a dict (or None) so this module stays config-package-agnostic.
891              self.watermark = self._normalize_watermark(watermark)
892     +        # Video encoder for the per-clip trim re-encode (#521). "libx264" (CPU) is the
default and
893     +        # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for
the 4K
894     +        # re-encode the watermark forces. The requested encoder is PROBED once per run
895     +        # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run
it.
896     +        self.encoder = (encoder or "libx264").strip() or "libx264"
897              os.makedirs(output_dir, exist_ok=True)
898
899          @staticmethod
900     @@ -225,6 +274,30 @@ class VideoCompiler:
901                  return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
902              return drawtext
903
904     +    def _select_encoder(self) -> Tuple[List[str], str]:
905     +        """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run,
plus the
906     +        chosen encoder name. Chosen ONCE (not per clip) so all clips share one codec
and the final
907     +        concat can stay ``-c copy``. NVENC (hardware) is the L4 win on the 4K re-
encode; if the
908     +        requested NVENC encoder can't run in this ffmpeg build (CI / non-GPU box), fall
back to
```

```
        libx264 so a misconfig never nukes the reel."""
        enc = self.encoder
        if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
            logger.warning(
                "[compile] encoder %r is not runnable in this ffmpeg build ? falling
back to "
                "libx264 (CPU). On the L4 worker this should be available.",
                enc,
            )
            enc = "libx264"
        if enc == "libx264":
            # Unchanged historical CPU settings (byte-compatible with the pre-#521
stitch).
            return ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"], enc
        # NVENC (H.264/HEVC): hardware encode. -cq is the CRF-equivalent constant-
quality knob; p4 is
        # a balanced speed/quality preset. yuv420p keeps the output broadly playable.
        return (
            ["-c:v", enc, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"],
            enc,
        )

     def _trim_one_segment(
         self,
         idx: int,
@@ -235,6 +308,7 @@ class VideoCompiler:
         tmp_dir: str,
         raw_video_path: str,
         watermark_filter: Optional[str],
+        video_codec_args: Optional[List[str]] = None,
     ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
         """Trim a single segment into its own clip file (extracted verbatim from the
         per-segment body of compile_highlights). Returns (clip_path, skip_record): on
@@ -297,6 +371,13 @@ class VideoCompiler:
             f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
(duration: {trim_duration:.2f}s)"
         )

+        # Default to libx264 (the historical settings) when a caller invokes
_trim_one_segment
+        # directly without resolving an encoder ? keeps this method's old behaviour
intact.
+        codec_args = (
+            video_codec_args
+            if video_codec_args is not None
+            else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
+        )
         base_cmd = [
             ffmpeg_bin(),
             "-y",
@@ -306,12 +387,7 @@ class VideoCompiler:
             str(round(padded_end, 3)),
             "-i",
             raw_video_path,
-            "-c:v",
-            "libx264",
-            "-preset",
-            "superfast",
-            "-crf",
-            "22",
+            *codec_args,
             "-c:a",
             "aac",
             "-b:a",
@@ -445,6 +521,11 @@ class VideoCompiler:
```

```
                watermark_filter = self._watermark_filter(tmp_dir)
                if watermark_filter:
                    logger.info("[watermark] applying branding overlay to each clip.")
    +           # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip
shares one codec
    +           # and the final concat stays -c copy (#521).
    +           video_codec_args, chosen_encoder = self._select_encoder()
    +           if chosen_encoder != "libx264":
    +               logger.info("[compile] encoding clips with %s (hardware).",
chosen_encoder)
                for idx, (raw_start, raw_end) in enumerate(segments):
                    clip_path, skip_record = self._trim_one_segment(
                        idx,
    @@ -455,6 +536,7 @@ class VideoCompiler:
                        tmp_dir,
                        raw_video_path,
                        watermark_filter,
    +                   video_codec_args,
                    )
                    if clip_path is not None:
                        temp_clips.append(clip_path)
    diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
    index 6d3420b..79db808 100644
    --- a/backend/worker/__main__.py
    +++ b/backend/worker/__main__.py
    @@ -419,6 +419,108 @@ def cmd_infer(args: argparse.Namespace) -> int:
         return 0


    +def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
    +    """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
    +    resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
    +    A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
    +    local = storage.fetch(
    +        spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
    +    )
    +    with open(local, encoding="utf-8") as f:
    +        data = json.load(f)
    +    if not isinstance(data, dict):
    +        raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
    +    return data
    +
    +
    +def _write_compile_marker(
    +    done_uri: str,
    +    result: dict,
    +    video_id: str,
    +    returncode: int,
    +    storage_cfg: dict,
    +    scratch: str,
    +) -> None:
    +    """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
    +    dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
    +    try:
    +        marker = {
    +            "video_id": video_id,
    +            "returncode": returncode,
    +            "status": result.get("status", "failed"),
    +            "download_url": result.get("download_url", ""),
    +            "reel_uri": result.get("reel_uri", ""),
```

```
1026    +            "message": result.get("message", ""),
1027    +        }
1028    +        local = os.path.join(scratch, "_compile_done.json")
1029    +        with open(local, "w", encoding="utf-8") as f:
1030    +            json.dump(marker, f)
1031    +        storage.put(local, done_uri, config=storage_cfg)
1032    +        logger.info("[worker] wrote compile marker -> %s", done_uri)
1033    +    except Exception as e:  # never fail a good stitch because the marker push hiccuped
1034    +        logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)
1035    +
1036    +
1037    +def cmd_compile(args: argparse.Namespace) -> int:
1038    +    """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU).
The stitch is the
1039    +    2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
1040    +    the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
1041    +    the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses
(so there is no
1042    +    forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
1043    +    bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
1044    +    from backend import orchestration
1045    +    from backend.config import StitchingConfig
1046    +
1047    +    config = load_config(args.config) if args.config else load_config()
1048    +    _apply_overrides(config, args.set)
1049    +    storage_cfg = config.storage.model_dump()
1050    +
1051    +    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
1052    +    os.makedirs(scratch, exist_ok=True)
1053    +    # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
1054    +    config.output.output_dir = scratch
1055    +    # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
value as a
1056    +    # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
1057    +    # destination even if the override were dropped.
1058    +    if not (config.storage.output_root or "").strip():
1059    +        config.storage.output_root = args.out_uri
1060    +
1061    +    spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
1062    +    video_id = str(spec.get("video_id") or "")
1063    +    out_name = str(spec.get("output_name") or "")
1064    +    source_ref = str(spec.get("video_uri") or "")
1065    +    locale = spec.get("locale")
1066    +    time_pairs = [
1067    +        (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
1068    +    ]
1069    +    stitching = StitchingConfig(**(spec.get("stitching") or {}))
1070    +
1071    +    print(
1072    +        f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
1073    +        f"encoder={stitching.encoder} out={out_name}"
1074    +    )
1075    +
1076    +    result = orchestration._stitch_reel(
1077    +        config,
1078    +        stitching,
1079    +        video_id,
1080    +        source_ref,
```

```
        time_pairs,
        out_name,
        locale,
        lambda stage: print(f"[worker] compile: {stage}"),
    )
    status = str(result.get("status") or "failed")
    rc = 0 if status == "success" else 3
    if status != "success":
        print(f"[worker] compile FAILED: {result.get('message')}")

    # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher passes it.
    if getattr(args, "done_uri", None):
        _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
    return rc


 #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter so a
 #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem collision).
 _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
@@ -760,6 +862,43 @@ def build_parser() -> argparse.ArgumentParser:
        )
        pi.set_defaults(func=cmd_infer)

    pc = sub.add_parser(
        "compile",
        help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged compile-spec",
    )
    pc.add_argument(
        "--spec-uri",
        required=True,
        help="storage URI of the control-plane-staged compile-spec JSON (time-pairs + stitching "
        "config + source uri + video_id)",
    )
    pc.add_argument(
        "--out-uri",
        default="",
        help="output ROOT (gs://? bucket); the reel is mirrored to highlights/<video_id>/<name>. "
        "Also forwarded via --set storage.output_root; either resolves the reel path.",
    )
    pc.add_argument(
        "--done-uri",
        default=None,
        help="storage URI for the compile completion MARKER (rc + download_url), so a remote "
        "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI stitch.",
    )
    pc.add_argument(
        "--config", default=None, help="config.json path (default: ./config.json)"
    )
    pc.add_argument(
        "--scratch", default=None, help="local scratch dir (default: a temp dir)"
    )
    pc.add_argument(
        "--set",
        action="append",
        default=[],
        metavar="k=v",
```

```
1137    +        help="config override, e.g. --set storage.output_root=gs://bucket",
1138    +    )
1139    +    pc.set_defaults(func=cmd_compile)
1140    +
1141         pt = sub.add_parser(
1142             "train",
1143             help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
1144
```

11. assistant (2026-07-08T06:04:38.154Z)

Let me read the surrounding context in orchestration.py to understand `_resolve_compile`, `reel_object_uri`, `_cloud_available`, and the download_url construction.

12. user (2026-07-08T06:04:39.654Z)

```
1040        row could change between submit and run). Verifies the video exists, the requested
segments match
1041        and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
1042        so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
1043        ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
1044        video = db.get_video(video_id)
1045        if not video:
1046            raise _CompileError(404, "Indexed video record not found.")
1047        # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
1048        try:
1049            out_name = safe_output_filename(output_filename)
1050        except ValueError as e:
1051            raise _CompileError(400, str(e)) from e
1052
1053        all_segments = db.query_play_segments(video_id, {})
1054        matched = [s for s in all_segments if s["id"] in segment_ids]
1055        if not matched:
1056            raise _CompileError(400, "No matching play segments found to stitch.")
1057
1058        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1059        kept = []
1060        for s in matched:
1061            meta = s.get("metadata") or {}
1062            sc = meta.get("shot_count") if isinstance(meta, dict) else None
1063            try:
1064                if sc is not None and int(sc) < stitching.min_shot_count:
1065                    continue
1066            except (TypeError, ValueError):
1067                pass  # unparseable count -> treat as unknown, keep
1068            kept.append(s)
1069        if len(kept) < len(matched):
1070            logger.info(
1071                "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
1072                stitching.min_shot_count,
1073                len(matched) - len(kept),
1074                len(matched),
1075            )
1076        if not kept:
1077            raise _CompileError(
1078                400,
1079                f"All {len(matched)} matching segments fall below "
1080                f"stitching.min_shot_count={stitching.min_shot_count}.",
1081            )
1082        return video, kept, out_name
```

```
1083
1084
1085    def reel_object_uri(cfg, video_id: str, name: str) -> str:
1086        """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
1087        the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
1088        mirror-upload and the /api/highlights redirect:
``<output_root>/highlights/<video_id>/<name>``."""
1089        root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1090        if not root or "://" not in root:
1091            return ""
1092        return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1093
1094
1095    def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1096        """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
1097        GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
1098        isn't silently constructed as a fresh default."""
1099        sc = getattr(cfg, "storage", None)
1100        return sc.model_dump() if sc is not None else None
1101
1102
1103    def _rendition_encode_profile(stitching: Any, rendition: str) -> Optional[Any]:
1104        if rendition != "sd":
1105            return None
1106        renditions = (
1107            stitching.get("renditions")
1108            if isinstance(stitching, dict)
1109            else getattr(stitching, "renditions", None)
1110        )
1111        if isinstance(renditions, dict):
1112            return renditions.get("sd")
1113        return getattr(renditions, "sd", None)
1114
1115
1116    def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1117        """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
1118        307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
1119        /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
1120        the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
1121        reel_uri = reel_object_uri(cfg, video_id, name)
1122        if not reel_uri:
1123            return None
1124        settings = _storage_settings(cfg)
1125        if storage.exists(reel_uri, config=settings):
1126            return storage.signed_url(reel_uri, config=settings, method="GET")
1127        return None
1128
1129
1130    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1131        """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1132        and return the {status, filepath, download_url} result the SPA polls for. Returns a
1133        ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1134        so the SPA always gets a clean verdict instead of a worker-crash error."""
1135        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
```

```

        notify = progress or (lambda stage: None)
        params = json.loads(job.get("params") or "{}")
        video_id = job.get("video_id") or ""
        segment_ids = params.get("segment_ids") or []
        locale = params.get("locale")
        rendition = str(params.get("rendition") or "original")
        if rendition not in {"original", "sd"}:
            return {
                "status": "failed",
                "message": f"Unsupported compile rendition: {rendition}",
                "video_id": video_id,
            }

        notify("resolving")
        try:
            video, kept, out_name = _resolve_compile(
                db, cfg, video_id, segment_ids, params.get("output_filename", "")
            )
        except _CompileError as e:
            return {"status": "failed", "message": e.detail, "video_id": video_id}

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        output_dir = (
            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
        )
        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
        # unknown/absent locale keeps the configured default unchanged.
        localized_watermark = localize_watermark(stitching.watermark, locale)
        encode_profile = _rendition_encode_profile(stitching, rendition)
        compiler = _main.VideoCompiler(
            output_dir,
            begin_padding=stitching.begin_padding,
            end_padding=stitching.end_padding,
            watermark=localized_watermark,
            encode_profile=encode_profile,
        )
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

        notify("stitching")
        logger.info(
            "[compile] stitching video_id=%s rendition=%s output=%s",
            video_id,
            rendition,
            out_name,
        )
        local_src = ""
        try:
            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
            local_src = storage.acquire_local_input(
                video["filepath"], getattr(cfg, "storage", None)
            )
            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
            with _LOCAL_COMPUTE_LANE:
                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
```

```
FAILURE, not a crash
1195            logger.error(
1196                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1197            )
1198            return {
1199                "status": "failed",
1200                "message": f"Highlight stitching failed: {e}",
1201                "video_id": video_id,
1202            }
1203        finally:
1204            if local_src:
1205                storage.cleanup_acquired_local_input(
1206                    video["filepath"], local_src, getattr(cfg, "storage", None)
1207                )
1208
1209        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1210        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1211        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1212        # still falls back to the local file on the same instance).
1213        reel_uri = reel_object_uri(cfg, video_id, out_name)
1214        if reel_uri:
1215            try:
1216                notify("uploading reel")
1217                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1218                # project/client) is used, not a fresh default.
1219                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1220            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1221                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1222
1223        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1224        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1225        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1226        notify("finished")
1227        return {
1228            "status": "success",
1229            "filepath": out_path,
1230            "download_url": download_url,
1231            "video_id": video_id,
1232            "filename": out_name,
1233            "rendition": rendition,
1234        }
1235
```

13. assistant (2026-07-08T06:04:56.024Z)

The working-tree `orchestration.py` shows a `rendition`-aware `_execute_compile` that differs from the PR diff. Let me check the actual repo state to review against the correct version.

14. user (2026-07-08T06:04:59.537Z)

```
bf97700facc2c7762a4cb2c7c039e7673eebab8b
---branch---
codex/issue-523-sd-reel-plan
---log---
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
```

```
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
---is dac6736 ancestor of HEAD?---
NO dac6736 not in HEAD
```

15. assistant (2026-07-08T06:05:10.961Z)

HEAD is at bf97700 on a different WIP branch (#523 SD-reel), so the working tree is NOT the #525 code. I must review against `dac6736`. Let me extract the post-change files at that commit.

16. user (2026-07-08T06:05:16.544Z)

```
   516 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/cloudrun_525.py
   626 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/compiler_525.py
   613 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/models_525.py
  1460 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/orch_525.py
   977 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/worker_525.py
  4192 total
```

17. user (2026-07-08T06:05:19.777Z)

```
1000                    "video_id": video_id,
1001                    "status": "failed",
1002                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
1003                    "results": [r.model_dump() for r in val_results],
1004                    "play_segments": [],
1005                }
1006                return response
1007
1008            segments = result.value if hasattr(result, "value") else result
1009            notify("finalizing")
1010            _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
1011
1012            response = {
1013                "video_id": video_id,
1014                "status": "success",
1015                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",  # type: ignore[arg-type]  # hasattr-unwrap above; Success arm has
.value
1016                "results": [r.model_dump() for r in val_results],
1017                "play_segments": segments,
1018            }
1019            return response
1020        finally:
1021            # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
1022            # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
1023            # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
1024            # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
1025            if isinstance(response, dict):
1026                response.setdefault(
1027                    "compute",
1028                    {
1029                        "path": compute_actual or "not_run",
1030                        "requested": getattr(req, "compute", None),
```

```
                    },
                )
            prov = response.get("compute", {}) if isinstance(response, dict) else {}
            logger.info(
                "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
                video_id,
                getattr(req, "compute", None),
                prov.get("path"),
                prov.get("dispatched"),
                (response or {}).get("status") if isinstance(response, dict) else None,
            )
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
            # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
            # doesn't surface arbitrary run signals.)
            paths = _main.emit_indexer_report(
                db=db,
                video_id=video_id,
                # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
                # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
                # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
                # still preserved in DB provenance via db.add_video.
                video_path=local_video,
                config=_report_config_for_input(config, req.video_path),
                val_results=val_results,
                val_context=val_context,
                segmenter_requested=req.segmenter,
                segmenter_actual=segmenter_actual,
                was_reingest=was_reingest,
                segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths
            storage.cleanup_acquired_local_input(
                req.video_path, local_video, getattr(config, "storage", None)
            )


    def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
        """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.

        Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
        missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
        to a no-op rather than blocking a job it cannot measure."""
        try:
            sc = (
                storage_config.model_dump()
                if hasattr(storage_config, "model_dump")
                else (storage_config or {})
            )
            backend = storage.get_storage(input_ref.backend, config=sc)
            return int(backend.stat(input_ref).size)
        except Exception as e:  # noqa: BLE001 - measuring is best-effort; inability must
not block
            logger.debug(
                "free-space: could not stat remote source %s: %s", input_ref.uri, e
            )
            return None
```

```
    class _CompileError(Exception):
        """A submit-time-validatable compile problem ? carries the HTTP status + detail so the API path
        maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job result."""

        def __init__(self, status_code: int, detail: str):
            super().__init__(detail)
            self.status_code = status_code
            self.detail = detail


    def _resolve_compile(
        db, cfg, video_id: str, segment_ids: List[int], output_filename: str
    ):
        """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-resolve, since a
        row could change between submit and run). Verifies the video exists, the requested segments match
        and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known shot_count are exempt
        so manual/legacy picks can't silently vanish), and the output name is traversal-safe. Raises
        ``_CompileError(404/400)``. Returns ``(video_row, kept_segments, safe_out_name)``."""
        video = db.get_video(video_id)
        if not video:
            raise _CompileError(404, "Indexed video record not found.")
        # Reject path traversal / absolute names (os.path.join would otherwise write anywhere on disk).
        try:
            out_name = safe_output_filename(output_filename)
        except ValueError as e:
            raise _CompileError(400, str(e)) from e

        all_segments = db.query_play_segments(video_id, {})
        matched = [s for s in all_segments if s["id"] in segment_ids]
        if not matched:
            raise _CompileError(400, "No matching play segments found to stitch.")

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
                pass  # unparseable count -> treat as unknown, keep
            kept.append(s)
        if len(kept) < len(matched):
            logger.info(
                "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the floor.",
                stitching.min_shot_count,
                len(matched) - len(kept),
                len(matched),
            )
        if not kept:
            raise _CompileError(
                400,
                f"All {len(matched)} matching segments fall below "
```

```
1144                f"stitching.min_shot_count={stitching.min_shot_count}.",
1145            )
1146        return video, kept, out_name
1147
1148
1149    def reel_object_uri(cfg, video_id: str, name: str) -> str:
1150        """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
1151        the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
1152        mirror-upload and the /api/highlights redirect:
``<output_root>/highlights/<video_id>/<name>``."""
1153        root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1154        if not root or "://" not in root:
1155            return ""
1156        return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1157
1158
1159    def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1160        """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
1161        GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
1162        isn't silently constructed as a fresh default."""
1163        sc = getattr(cfg, "storage", None)
1164        return sc.model_dump() if sc is not None else None
1165
1166
1167    def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1168        """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
1169        307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
1170        /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
1171        the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
1172        reel_uri = reel_object_uri(cfg, video_id, name)
1173        if not reel_uri:
1174            return None
1175        settings = _storage_settings(cfg)
1176        if storage.exists(reel_uri, config=settings):
1177            return storage.signed_url(reel_uri, config=settings, method="GET")
1178        return None
1179
```

18. user (2026-07-08T06:05:25.552Z)

```
1179
1180
1181    def _compile_wait_message(elapsed_sec: float) -> str:
1182        """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
1183        ``_wait_progress_message``). Minute-granular; under a minute reads as cold-start."""
1184        minutes = int(elapsed_sec) // 60
1185        if minutes < 1:
1186            return "stitching on cloud GPU (NVENC) ? starting up"
1187        return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
1188
1189
1190    def _stitch_reel(
1191        cfg,
1192        stitching,
1193        video_id: str,
1194        source_ref: str,
1195        time_pairs: List[Tuple[float, float]],
```

```
1196        out_name: str,
1197        locale: Optional[str],
1198        notify,
1199    ) -> Dict[str, Any]:
1200        """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1201        (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1202        needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1203        with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1204        trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1205        returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1206        ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
1207        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
1208
1209        output_dir = (
1210            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1211        )
1212        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1213        # unknown/absent locale keeps the configured default unchanged.
1214        localized_watermark = localize_watermark(stitching.watermark, locale)
1215        compiler = _main.VideoCompiler(
1216            output_dir,
1217            begin_padding=stitching.begin_padding,
1218            end_padding=stitching.end_padding,
1219            watermark=localized_watermark,
1220            encoder=getattr(stitching, "encoder", "libx264"),
1221        )
1222
1223        notify("stitching")
1224        local_src = ""
1225        try:
1226            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1227            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1228            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1229            local_src = storage.acquire_local_input(
1230                source_ref, getattr(cfg, "storage", None)
1231            )
1232            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1233            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1234            with _LOCAL_COMPUTE_LANE:
1235                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1236        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1237            logger.error(
1238                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1239            )
1240            return {
1241                "status": "failed",
1242                "message": f"Highlight stitching failed: {e}",
1243                "video_id": video_id,
1244            }
1245        finally:
```

```
            if local_src:
                storage.cleanup_acquired_local_input(
                    source_ref, local_src, getattr(cfg, "storage", None)
                )

        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
        # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
        # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
        reel_uri = reel_object_uri(cfg, video_id, out_name)
        if reel_uri:
            try:
                notify("uploading reel")
                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
                # project/client) is used, not a fresh default.
                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)

        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
        notify("finished")
        return {
            "status": "success",
            "filepath": out_path,
            "download_url": download_url,
            "video_id": video_id,
            "reel_uri": reel_uri or "",
        }


    def _dispatch_compile_remote(
        cfg,
        db,
        video_id: str,
        segment_ids: List[int],
        output_filename: str,
        locale: Optional[str],
        notify,
    ) -> Dict[str, Any]:
        """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
        control-plane owns the DB, so it RESOLVES the reel's segments here
(``_resolve_compile``), stages a
        small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
        to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
        done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
        worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
        ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
        from backend.compute import CompileJobSpec, get_compute_backend

        notify("resolving")
```

```
        try:
            video, kept, out_name = _resolve_compile(
                db, cfg, video_id, segment_ids, output_filename
            )
        except _CompileError as e:
            return {"status": "failed", "message": e.detail, "video_id": video_id}

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        storage_cfg = getattr(cfg, "storage", None)
        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
        compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
        if compute is None:
            # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
            # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ?
a slow reel
            # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
            logger.warning(
                "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
                "stitching in-process on the control-plane instead.",
                out_root,
            )
            time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
            return _stitch_reel(
                cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
            )

        # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe
on the wire);
        # the stitching config is the control-plane's RESOLVED one, with the encoder swapped
to the L4's
        # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes
the reel.
        stitch_dump = stitching.model_dump()
        stitch_dump["encoder"] = getattr(
            getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
        )
        spec_payload = {
            "video_id": video_id,
            "video_uri": video["filepath"],
            "output_name": out_name,
            "locale": locale,
            "time_pairs": [
                [float(s["start_time"]), float(s["end_time"])] for s in kept
            ],
            "stitching": stitch_dump,
        }
        token = uuid.uuid4().hex
        spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
        try:
            notify("staging compile spec")
            with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
                local_spec = os.path.join(td, "spec.json")
                with open(local_spec, "w", encoding="utf-8") as f:
                    json.dump(spec_payload, f)
                storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
        except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
            logger.error(
                "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
```

```
            )
            return {
                "status": "failed",
                "message": f"cloud compile: could not stage the compile spec: {e}",
                "video_id": video_id,
            }

        # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
        # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
        overrides = [f"storage.output_root={out_root}"]
        backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
        if backend_name:
            overrides.append(f"storage.backend={backend_name}")
        spec = CompileJobSpec(
            out_uri=out_root,
            spec_uri=spec_uri,
            video_id=video_id,
            output_name=out_name,
            config_overrides=tuple(overrides),
        )

        notify("dispatching stitch (cloud_run)")
        logger.info(
            "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
            video_id,
            out_root,
            len(kept),
        )
        try:
            result = compute.run_compile(
                spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
            )
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
            return {
                "status": "failed",
                "message": f"cloud compile dispatch error: {e}",
                "video_id": video_id,
            }
        if result.returncode != 0:
            msg = str(
                result.report.get("message")
                or f"cloud compile job failed (returncode {result.returncode})."
            )
            return {"status": "failed", "message": msg, "video_id": video_id}

        download_url = str(
            result.report.get("download_url")
            or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
        )
        logger.info(
            "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
            result.execution,
            result.outputs_uri,
            video_id,
        )
        notify("finished")
        return {
            "status": "success",
            "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
            "download_url": download_url,
```

```
1413            "video_id": video_id,
1414            "compute": {
1415                "path": "cloud_run",
1416                "execution": result.execution,
1417                "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
1418                "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
1419            },
1420        }
1421
1422
1423    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1424        """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
1425        the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
1426        result for an EXPECTED problem (missing video / no segments / a stitch error) so the
SPA always gets
1427        a clean verdict instead of a worker-crash error.
1428
1429        #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``)
and cloud
1430        dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
1431        (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
1432        2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process
here, unchanged."""
1433        notify = progress or (lambda stage: None)
1434        params = json.loads(job.get("params") or "{}")
1435        video_id = job.get("video_id") or ""
1436        segment_ids = params.get("segment_ids") or []
1437        locale = params.get("locale")
1438        output_filename = params.get("output_filename", "")
1439
1440        if (
1441            resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
1442            and _cloud_available(cfg)
1443        ):
1444            return _dispatch_compile_remote(
1445                cfg, db, video_id, segment_ids, output_filename, locale, notify
1446            )
1447
1448        notify("resolving")
1449        try:
1450            video, kept, out_name = _resolve_compile(
1451                db, cfg, video_id, segment_ids, output_filename
1452            )
1453        except _CompileError as e:
1454            return {"status": "failed", "message": e.detail, "video_id": video_id}
1455
1456        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1457        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1458        return _stitch_reel(
1459            cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
1460        )
```

19. assistant (2026-07-08T06:06:29.196Z)

Let me read the full `_maybe_dispatch_remote`, `_cloud_available`, and `_execute_processing` validation section at dac6736.

20. user (2026-07-08T06:06:32.168Z)

```
195            )
196        return bool(path) and os.path.exists(os.path.expanduser(path))
```

```
197
198
199     def _cloud_available(cfg) -> bool:
200         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
201
202         True when the control plane is wired for Cloud Run: either the configured default
target is
203         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
204         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
205         are present so a forced per-request override can actually dispatch + collect a
result.
206
207         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
208         deployment = getattr(cfg, "deployment", None)
209         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
210             return True
211         has_job = bool(
212             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
213         )
214         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
215         return bool(has_job and out_root)
216
217
218     def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
219         """Would DETECT for this request run on the L4 worker (vs in-process)? Mirrors the
220         ``_maybe_dispatch_remote`` decision WITHOUT side effects (#521), so
``_execute_processing`` can
221         decide whether to DELEGATE validation to the worker
(``phase_placement.validation``). A per-request
222         picker choice wins; otherwise ``phase_placement.segment`` (whose legacy default is
cloud iff
223         ``compute_target==cloud_run``) gates it, and cloud must actually be available. A
pre-dispatch reject
224         (e.g. cloud requested but unconfigured) counts as NOT-on-cloud ? nothing runs
remotely."""
225         choice = (req_compute or "").strip().lower()
226         if choice == "local":
227             return False
228         if choice == "cloud":
229             return _cloud_available(cfg)
230         return (
231             resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
232             and _cloud_available(cfg)
233         )
234
235
236     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
237         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
238         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
239         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
240         (queue/cold-start), so the very first tick already shows movement."""
241         minutes = int(elapsed_sec) // 60
242         if minutes < 1:
243             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
244         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
245
246
247     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
248         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
```

```
gs://
        object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
        deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
        ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
        if getattr(input_ref, "backend", "") != "gcs":
            return None
        sc = _storage_settings(cfg)
        if not sc:
            return None
        try:
            st = storage.get_storage("gcs", config=sc).stat(input_ref)
            return getattr(st, "md5", None) or None
        except Exception:  # noqa: BLE001 ? best-effort identity, the drain still hashes
            logger.info(
                "[API] submit-time md5 unavailable for %s (metadata stat failed)",
                getattr(input_ref, "uri", input_ref),
                exc_info=True,
            )
            return None


    def _cached_process_payload(
        video_id: str, validation_results, segments, requested_compute
    ) -> Dict[str, Any]:
        """The /api/process result for a video whose detection already exists ? ONE shape
for
        both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
        return {
            "video_id": video_id,
            "status": "success",
            "message": (
                f"Reused {len(segments)} cached play segments. "
                "Pass force=true to reprocess this video."
            ),
            "results": validation_results or [],
            "play_segments": segments,
            "cached": True,
            "compute": {"path": "cached", "requested": requested_compute},
        }


    def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
        """Fetch + parse one small JSON object (cloud backends require a real dst ? the
        ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
        ref = storage.parse_uri(uri)
        with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
            local = storage.fetch(
                uri, os.path.join(td, ref.name or "report.json"), config=storage_config
            )
            with open(local, encoding="utf-8") as f:
                loaded = json.load(f)
        return loaded if isinstance(loaded, dict) else None


    # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
    # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
    # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
    # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
    _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
```

```
306     _REHYDRATE_LOCKS_GUARD = threading.Lock()
307
308
309     def _rehydrate_lock(video_id: str) -> threading.Lock:
310         with _REHYDRATE_LOCKS_GUARD:
311             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
312
313
314     def probe_process_cache(db, cfg, video_id: str) -> bool:
```

21. assistant (2026-07-08T06:06:35.508Z)

Now the critical `_maybe_dispatch_remote` full body to compare against `_will_detect_on_cloud`.

22. user (2026-07-08T06:06:38.725Z)

```
505             )
506             choice = ""
507         if choice == "local":
508             # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
509             # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
510             # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
511             # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
512             # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
513             # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
514             # below exactly as before (the weights-present check is the escape hatch).
515             if (
516                 (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
517                 and _cloud_available(cfg)
518                 and not _local_detector_weights_present(cfg, seg_name)
519             ):
520                 return _failed(
521                     f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
522                     "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
523                     False,
524                 )
525             return None  # explicit "run on my machine" ? in-process regardless of the
server's target
526         if choice == "cloud":
527             if not _cloud_available(cfg):
528                 return _failed(
529                     "cloud-serving was requested but this server isn't configured for it "
530                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
531                     False,
532                 )
533             compute = get_compute_backend(cfg, target_override="cloud_run")
534         else:
535             # No explicit picker ? consult phase_placement.segment (#521). Its legacy
default is
536             # cloud_run_l4 IFF compute_target==cloud_run, so this preserves today's
behaviour; setting
537             # phase_placement.segment relocates DETECT by config alone (e.g. force it back
on the
538             # control-plane, or onto the L4 on a default-local box wired for cloud).
539             if (
540                 resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
```

```
                and _cloud_available(cfg)
            ):
                compute = get_compute_backend(cfg, target_override="cloud_run")
            else:
                compute = None
        if compute is None:
            return None  # default-OFF / placed on control_plane ? the in-process segmenter
below

        storage_cfg = getattr(cfg, "storage", None)

        # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
        try:
            input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
        except ValueError as e:
            return _failed(
                f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
            )
        if input_ref.backend == "local":
            # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
            # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
            # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
            # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
            # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
            # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
            # default, while a bare/local-looking root can't stage no matter what
input_backend says
            # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
            # gives us nothing to build the staged URI under).
            input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
            try:
                root_backend = (
                    storage.parse_uri(input_root).backend if input_root else "local"
                )
            except ValueError:  # unsupported scheme in input_root ? nowhere real to stage
to
                root_backend = "local"
            if root_backend == "local":
                return _failed(
                    "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
                    "stage local uploads into; a path local to this machine can't be read by
the Cloud "
                    "Run job.",
                    False,
                )
            local_src = input_ref.key  # resolved local path of the upload
            staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
            try:
                notify("staging upload to bucket")
                # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
                # region/addressing for R2·DO·MinIO; GCS project/client) ? not a fresh
default client.
                storage.put(
                    local_src,
```

```
590                     staged_uri,
591                     config=(
592                         storage_cfg.model_dump() if storage_cfg is not None else None
593                     ),
594                 )
595             except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
596                 logger.error(
597                     "[API] cloud-serving: failed to stage %s -> %s: %s",
598                     local_src,
599                     staged_uri,
600                     e,
601                     exc_info=True,
602                 )
603                 return _failed(
604                     f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
605                     False,
606                 )
607             # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
608             # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
609             # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
610             req.video_path = staged_uri
611             input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
612             db.add_video(
613                 video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
614             )
615         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
616         if not out_root:
617             return _failed(
618                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
619                 "? that's where the Cloud Run job writes the result.",
620                 False,
621             )
622
623         notify("dispatching (cloud_run)")
624         logger.info(
625             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
626             req.video_path,
627             out_root,
628         )
629         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
630         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
631         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
632         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
633         overrides = []
634         idx_cfg = getattr(cfg, "indexing", None)
635         sk = getattr(idx_cfg, "skip_ai_handoff", None)
636         if sk is not None:
637             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
638         # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
639         # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
640         # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
641         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
```

```
decode_backend is
642         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
643         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
644         wasb_cfg = getattr(idx_cfg, "wasb", None)
645         if wasb_cfg is not None:
646             decode_backend = getattr(wasb_cfg, "decode_backend", None)
647             if decode_backend:
648                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
649             batch_size = getattr(wasb_cfg, "batch_size", None)
650             if batch_size:
651                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
652             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
653             if prefetch:
654                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
655         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
656         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
657         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
658         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
659         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
660         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
661         # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the
control-plane is
662         # the authoritative gate ? it already validated with its resolved config (only a
PASS reaches
663         # here), so the worker SKIPS its redundant re-validation (#513). If validation is
placed on the
664         # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators ?
see
665         # _execute_processing), so the worker must NOT skip: it becomes the gate.
666         worker_skips_validation = (
667             resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
668         )
669         spec = ComputeJobSpec(
670             video_uri=req.video_path,
671             out_uri=out_root,
672             segmenter=seg_name,
673             config_overrides=tuple(overrides),
674             skip_validation=worker_skips_validation,
675         )
676         try:
677             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
678             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
679             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
680             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
681             result = compute.run_infer(
682                 spec,
683                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
684             )
685         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
686             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
687             return _failed(f"cloud dispatch error: {e}", True)
688         if result.returncode != 0:
689             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
690         if result.video_id and result.video_id != video_id:
691             # The worker re-hashes the bucket bytes; a divergence means the input changed
```

```
between this
692             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
693             logger.warning(
694                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
695                 "and the worker hash; ingesting under the local id.",
696                 result.video_id,
697                 video_id,
698             )
699
700         notify("ingesting (cloud_run)")
701         storage_dict = (
702             storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
703             if hasattr(storage_cfg, "model_dump")
704             else {}
705         )
706         try:
707             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
708         except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
709             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
710             return _failed(
711                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
712                 True,
713             )
714
```

23. assistant (2026-07-08T06:07:04.945Z)

Let me read `get_compute_backend` to check whether it can return None while `_cloud_available` is True (a validation-delegation gap).

24. user (2026-07-08T06:07:09.146Z)

```
     1  """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
     2
     3  COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
     4  ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
     5  otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
     6  identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
     7  changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
     8  """
     9
    10  from __future__ import annotations
    11
    12  import dataclasses
    13  import os
    14  from typing import Any, Optional
    15
    16  from backend.compute.base import (
    17      CompileJobSpec,
    18      ComputeBackend,
    19      ComputeJobSpec,
    20      ComputeResult,
    21      RemoteExecutionFailed,
    22  )
    23  from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
    24
    25  __all__ = [
```

```
    "ComputeBackend",
    "ComputeJobSpec",
    "CompileJobSpec",
    "ComputeResult",
    "RemoteExecutionFailed",
    "get_compute_backend",
]


def _storage_dict(config: Any) -> dict:
    storage = getattr(config, "storage", None)
    if storage is None:
        return {}
    if hasattr(storage, "model_dump"):
        return storage.model_dump()  # type: ignore[no-any-return]
    return dict(storage) if isinstance(storage, dict) else {}


def get_compute_backend(
    config: Any,
    *,
    run_client: Optional[Any] = None,
    policy: ExecutionPolicy = DEFAULT_POLICY,
    token: Optional[str] = None,
    target_override: Optional[str] = None,
) -> Optional[ComputeBackend]:
    """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
    in-process (the default-OFF path).

    Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
    Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
    / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
    pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).

    ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
    per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
    server's configured default. ``None`` falls back to
``config.deployment.compute_target``.
    """
    target = (
        target_override
        if target_override is not None
        else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
    )
    if target != "cloud_run":
        return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)

    # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
    from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient

    # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
    # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
so a
    # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild §4.3; the
old
    # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps
an
```

```
    79      # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
    80      if policy is DEFAULT_POLICY:
    81          max_wait = getattr(
    82              getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
    83          )
    84          if max_wait:
    85              policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
    86
    87      job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
    88      region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
    89      project = os.environ.get("GOOGLE_CLOUD_PROJECT")
    90      client = run_client if run_client is not None else GoogleCloudRunJobClient()
    91      return CloudRunCompute(
    92          run_client=client,
    93          job_name=job_name,
    94          region=region,
    95          project=project,
    96          storage_config=_storage_dict(config),
    97          policy=policy,
    98          token=token,
    99      )
```

25. assistant (2026-07-08T06:07:22.000Z)

Let me read the top of `_maybe_dispatch_remote` (choice normalization) and `_execute_processing`'s validation block in full.

26. user (2026-07-08T06:07:25.252Z)

```
470         from backend.compute import ComputeJobSpec, get_compute_backend
471
472         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
473             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
474             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
475             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
476             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
477             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
478             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
479             # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
480             return (
481                 {
482                     "video_id": video_id,
483                     "status": "failed",
484                     "message": msg,
485                     "results": [r.model_dump() for r in val_results],
486                     "play_segments": [],
487                     "compute": {
488                         "path": "cloud_run" if ran else "not_run",
489                         "requested": req.compute,
490                         "target": "cloud_run",
491                         "dispatched": ran,
492                     },
493                 },
494                 ran,
495             )
496
497         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
498         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
```

```
499         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
500         choice = (req.compute or "").strip().lower()
501         if choice and choice not in ("local", "cloud"):
502             logger.warning(
503                 "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
504                 req.compute,
505             )
506             choice = ""
507         if choice == "local":
508             # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
509             # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
```

27. assistant (2026-07-08T06:07:34.257Z)

Let me read the full `_execute_processing` validation region and where `val_results` is initialized.

28. user (2026-07-08T06:07:35.755Z)

```
820         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
821         compute_actual: Optional[str] = None
822         response: Optional[Dict[str, Any]] = None
823         try:
824             if (
825                 existing_video
826                 and existing_video.get("status") == "processed"
827                 and not req.force
828             ):
829                 cached_segments = db.query_play_segments(video_id, {})
830                 if cached_segments:
831                     logger.info(
832                         "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
833                         video_id,
834                     )
835                     response = _cached_process_payload(
836                         video_id,
837                         existing_video.get("validation_results", []),
838                         cached_segments,
839                         getattr(req, "compute", None),
840                     )
841                     return response
842
843             # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
844             #    UNLESS this exact content already has a cached verdict under the current
validator config.
845             notify("validating")
846             # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
847             # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
848             # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
849             # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
850             # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
851             # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
```

```
852             # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
853             # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
854             # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
855             # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
856             # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
857             #
858             # #521 phase_placement.validation: when validation is placed on the L4 AND
detection will
859             # actually dispatch there, the control-plane DELEGATES validation to the worker
? it does NOT
860             # run the validator suite here (the whole point is to move the CPU-bound
validation OFF the
861             # 2-vCPU control-plane). The worker then validates (skip_validation=False) and
rejects a bad
862             # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed job.
When detection
863             # is NOT on the cloud (in-process), the control-plane always validates ? there
is no worker to
864             # delegate to (the config guard makes validation=cloud_run_l4 require a cloud
segment).
865             validate_on_control_plane = (
866                 resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
867                 or not _will_detect_on_cloud(config, getattr(req, "compute", None))
868             )
869             if not validate_on_control_plane:
870                 logger.info(
871                     "[API] validation delegated to the L4 worker "
872                     "(deployment.phase_placement.validation=cloud_run_l4) ? the control-
plane skips its "
873                     "validator suite; the worker is the gate for %s.",
874                     req.video_path,
875                 )
876             val_fingerprint = (
877                 registry.config_fingerprint(config) if validate_on_control_plane else ""
878             )
879             # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
880             # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
881             # failing the job. The cache is a pure speedup ? never a new failure mode.
882             reused_validation = False
883             if validate_on_control_plane and not req.force:
884                 try:
885                     cached = db.get_validation_cache(video_id)
886                     if cached is not None and cached.get("fingerprint") == val_fingerprint:
887                         # Replay the stored ValidationResult list; val_context stays {} (no
fresh
888                         # ffprobe_meta ? the report's media block degrades gracefully to
size-only, the
889                         # deliberate cost of skipping the decode). Segmentation reads
neither of these.
890                         val_results = [
891                             ValidationResult(**r) for r in cached.get("results", [])
892                         ]
893                         reused_validation = True
894                         logger.info(
895                             "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
896                             "validator config unchanged ? skipping the validator suite. "
897                             "force=true re-validates.",
```

```
898                             cached.get("passed"),
899                             video_id,
900                         )
901                 except Exception:  # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
902                     reused_validation = False
903                     logger.warning(
904                         "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
905                         video_id,
906                         exc_info=True,
907                     )
908             if validate_on_control_plane and not reused_validation:
909                 logger.info(f"Running validations for {req.video_path}...")
910                 val_results, val_context = registry.run_all_with_context(local_video,
config)
911             failures = [r for r in val_results if not r.passed]
912             if validate_on_control_plane and not reused_validation:
913                 # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
914                 # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
915                 db.set_validation_cache(
916                     video_id,
917                     val_fingerprint,
918                     not failures,
919                     [r.model_dump() for r in val_results],
920                 )
921
922             if failures:
923                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
924                 # status; it no longer destroys the video's prior good segments.
925                 db.add_video(
926                     video_id,
927                     req.video_path,
928                     "failed",
929                     [r.model_dump() for r in val_results],
930                 )
931                 response = {
932                     "video_id": video_id,
933                     "status": "failed",
934                     "message": _validation_failure_message(failures),
935                     "results": [r.model_dump() for r in val_results],
936                 }
937                 return response
938
939             # 2. Run segmenter & indexer
940             logger.info("[API] Video passed checks. Segmenting and indexing...")
941             db.add_video(
942                 video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
943             )
944
945             segmenter_cls = segmenter_registry.get(seg_name)
946             if not segmenter_cls:
947                 # Submit-time validation already 400s unknown names; this is the defensive
948                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
949                 available = list(segmenter_registry.list_available().keys())
```

29. user (2026-07-08T06:07:44.985Z)

```
816:    val_results: List[Any] = []
890:                     val_results = [
1055:            val_results=val_results,
```

30. assistant (2026-07-08T06:07:49.837Z)

Let me read `cloud_run.py` `run_infer` to verify the detect marker prefix (`_dispatch/`) and compare the two paths.

31. user (2026-07-08T06:07:51.711Z)

```
230             self._client = run_client
231             self._job_name = job_name
232             self._region = region
233             self._project = project
234             self._storage_config = storage_config or {}
235             self._policy = policy
236             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
237             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
238             self._token = token
239             self._container_overrides = (
240                 _DEFAULT_CONTAINER_OVERRIDES
241                 if container_overrides is None
242                 else tuple(container_overrides)
243             )
244
245         def run_infer(
246             self,
247             spec: ComputeJobSpec,
248             *,
249             on_wait: "Callable[[float], None] | None" = None,
250         ) -> ComputeResult:
251             out_root = spec.out_uri.rstrip("/")
252             token = self._token or uuid.uuid4().hex
253             done_uri = f"{out_root}/_dispatch/{token}.done"
254             argv: List[str] = [
255                 "infer",
256                 "--video-uri",
257                 spec.video_uri,
258                 "--out-uri",
259                 spec.out_uri,
260                 "--done-uri",
261                 done_uri,
262             ]
263             if spec.segmenter:
264                 argv += ["--segmenter", spec.segmenter]
265             if spec.skip_validation:
266                 # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
267                 # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
268                 argv.append("--skip-validation")
269             for kv in (*spec.config_overrides, *self._container_overrides):
270                 argv += ["--set", kv]
271
272             execution = self._client.run_job(
273                 self._job_name, argv, region=self._region, project=self._project
274             )
275             LOG.info(
276                 "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
277                 self._job_name,
278                 execution,
279                 done_uri,
280             )
281
282             marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
infer")
```

```
283             video_id = str(marker.get("video_id", ""))
284             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
285             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
286             rc = int(marker.get("returncode", 1))
287             if not video_id:
288                 LOG.warning(
289                     "cloud-run: job=%s completion marker present but empty/unreadable (no "
290                     "video_id) ? treating as failure",
291                     self._job_name,
292                 )
293                 rc = rc or 1
294             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
295             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
296             LOG.info(
297                 "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
298             )
299             return ComputeResult(
300                 returncode=rc,
301                 outputs_uri=outputs_uri,
302                 video_id=video_id,
303                 report=report,
304                 execution=str(execution or ""),
305             )
306
307         def run_compile(
308             self,
309             spec: "CompileJobSpec",
310             *,
311             on_wait: "Callable[[float], None] | None" = None,
312         ) -> ComputeResult:
313             """Dispatch a reel-COMPILE (#521): run ``python -m backend.worker compile`` on
the L4 (NVENC),
314             bucket-poll the done-marker, and return the worker's rc + marker. Reuses the
EXACT
315             dispatch?poll?marker machinery as ``run_infer`` (``_await_marker`` ? crash-fast
+ timeout
316             rescue), so a stitch that crashes or overruns fails the same way a detection
does ? not a
317             ~65-min opaque hang."""
318             out_root = spec.out_uri.rstrip("/")
319             token = self._token or uuid.uuid4().hex
320             # Distinct marker prefix from infer's _dispatch/ so a compile + a detection for
the same
321             # bucket can never collide on a marker.
322             done_uri = f"{out_root}/_compile/{token}.done"
323             argv: List[str] = [
324                 "compile",
325                 "--out-uri",
326                 spec.out_uri,
327                 "--spec-uri",
328                 spec.spec_uri,
329                 "--done-uri",
330                 done_uri,
331             ]
332             # Compile carries NO container-inline overrides: cmd_compile never consults
compute_target and
333             # never re-dispatches, so the only overrides are the CP-forwarded
storage/encoder knobs.
334             for kv in spec.config_overrides:
335                 argv += ["--set", kv]
336
337             execution = self._client.run_job(
338                 self._job_name, argv, region=self._region, project=self._project
```

```
339             )
340             LOG.info(
341                 "cloud-run: dispatched COMPILE job=%s exec=%s; bucket-polling %s",
342                 self._job_name,
343                 execution,
344                 done_uri,
345             )
346             marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
compile")
347             video_id = str(marker.get("video_id", "") or spec.video_id)
348             rc = int(marker.get("returncode", 1))
349             LOG.info(
350                 "cloud-run: COMPILE job=%s done (video_id=%s rc=%d)",
351                 self._job_name,
352                 video_id,
353                 rc,
354             )
355             # The marker carries the worker's stitch result (download_url / reel_uri /
status); surface it
356             # as ``report`` so the dispatcher returns the same {status, filepath,
download_url} shape the
357             # SPA expects. ``outputs_uri`` is the mirrored reel URI when present.
358             return ComputeResult(
359                 returncode=rc,
360                 outputs_uri=str(marker.get("reel_uri", "") or ""),
361                 video_id=video_id,
362                 report=marker,
363                 execution=str(execution or ""),
364             )
365
366         def _await_marker(
367             self,
368             done_uri: str,
369             execution: object,
370             on_wait: "Callable[[float], None] | None",
371             *,
372             label: str,
373         ) -> dict:
374             """Bucket-poll ``done_uri`` to a completion marker ? shared by ``run_infer`` and
375             ``run_compile``. ``on_wait`` is the #482 live heartbeat (each still-waiting tick
reaches the
376             caller so job.progress moves). A ``RemoteExecutionFailed`` (the fast negative
path in
377             ``_poll_check``) is NOT a ``PolicyTimeout`` so it propagates uncaught ? the
dispatch handler
378             maps it to a distinct failure; a genuine poll TIMEOUT is handled (rescue-or-
cancel)."""
379             try:
380                 return poll_until(
381                     lambda: self._poll_check(done_uri, str(execution or "")),
382                     self._policy,
383                     label=label,
384                     logger=LOG,
385                     on_tick=on_wait,
386                 )
387             except PolicyTimeout as timeout_exc:
388                 return self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
389
```

32. user (2026-07-08T06:08:00.352Z)

```
389
390         def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
391             """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
```

```
392
393             The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
394             wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
395             FAILURE* marker resolves through the normal path with its real returncode. Only
when the
396             marker is absent do we consult the execution status: a worker that TERMINATED
without ever
397             writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
398             otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
399             opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
400             failure.
401
402             Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
403             ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,
no marker
404             will ever appear)."""
405             marker = self._read_marker(done_uri)
406             if marker is not None:
407                 return marker  # success signal ? authoritative even if status later flips
408             state = self._terminal_execution_state(execution)
409             if state is None:
410                 return None  # still pending/running, or status unknown ? keep bucket-
polling
411             # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
412             # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
413             # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
414             # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
415             marker = self._read_marker(done_uri)
416             if marker is not None:
417                 return marker
418             raise RemoteExecutionFailed(
419                 f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
420                 f"terminal state {state!r} with NO completion marker ? the worker exited
before "
421                 "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
422                 "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
423                 "inspect the Cloud Run Job execution log to diagnose: "
424                 f"`gcloud run jobs executions describe {execution or '<execution>'} "
425                 f"--region {self._region}`."
426             )
427
428         def _terminal_execution_state(self, execution: str) -> Optional[str]:
429             """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
430             flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
431             so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
432             ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
433             if not execution:
434                 return None
435             try:
436                 return self._client.execution_terminal_state(execution)
```

```
            except Exception:  # noqa: BLE001 ? advisory probe: never let it kill the bucket-poll
                LOG.debug(
                    "cloud-run: execution status probe failed for %s ? treating as unknown",
                    execution,
                    exc_info=True,
                )
                return None

        def _handle_poll_timeout(
            self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
        ) -> dict:
            """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.

            The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed job's own
            ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past the deadline:
            ONE final marker check rescues that completed GPU run instead of wasting it. If the marker
            is genuinely absent, the execution is still running ? best-effort cancel it (otherwise the
            L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout`` carrying the
            execution name so the operator can verify the state in the GCP console. A cancel failure
            NEVER masks the original timeout."""
            try:
                late = self._read_marker(done_uri)
            except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence ? failure
                LOG.warning(
                    "cloud-run: final post-timeout marker check failed ? treating as absent",
                    exc_info=True,
                )
                late = None
            if late is not None:
                LOG.warning(
                    "cloud-run: job=%s exec=%s done-marker found on the final post-timeout check "
                    "? rescuing the completed result",
                    self._job_name,
                    execution,
                )
                return late
            try:
                self._client.cancel_execution(execution)
            except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the timeout
                cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
                LOG.warning(
                    "cloud-run: best-effort cancel of execution=%s failed: %s",
                    execution,
                    cancel_exc,
                    exc_info=True,
                )
            else:
                cancel_note = "cancel requested"
                LOG.info("cloud-run: requested cancel of execution=%s", execution)
            raise PolicyTimeout(
                f"{timeout_exc} [cloud-run job={self._job_name} "
                f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution "
                "state in the GCP console]"
```

```
490             ) from timeout_exc
491
492         # --- storage-backed completion signal (bucket-poll) ---
493         def _read_marker(self, done_uri: str) -> Optional[dict]:
494             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
495             ref = parse_uri(done_uri)
496             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
497                 return None
498             return (
499                 self._read_json(done_uri) or {}
500             )  # empty dict still terminates the poll (present == done)
501
502         def _read_json(self, uri: str) -> dict:
503             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
504             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
505             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
506             try:
507                 ref = parse_uri(uri)
508                 with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
509                     dst = os.path.join(td, ref.name or "obj.json")
510                     local = fetch(uri, dst, config=self._storage_config)
511                     with open(local, "r", encoding="utf-8") as f:
512                         data: Any = json.load(f)
513                 return data if isinstance(data, dict) else {}
514             except Exception:
515                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
516                 return {}
```

33. assistant (2026-07-08T06:08:28.265Z)

Let me read the worker `cmd_compile`, its spec round-trip, and the argument/main dispatch.

34. user (2026-07-08T06:08:31.452Z)

```
419         return 0
420
421
422     def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
423         """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
424         resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
425         A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
426         local = storage.fetch(
427             spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
428         )
429         with open(local, encoding="utf-8") as f:
430             data = json.load(f)
431         if not isinstance(data, dict):
432             raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
433         return data
434
435
436     def _write_compile_marker(
437         done_uri: str,
438         result: dict,
439         video_id: str,
440         returncode: int,
441         storage_cfg: dict,
442         scratch: str,
```

```
    ) -> None:
        """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
        dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "status": result.get("status", "failed"),
                "download_url": result.get("download_url", ""),
                "reel_uri": result.get("reel_uri", ""),
                "message": result.get("message", ""),
            }
            local = os.path.join(scratch, "_compile_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote compile marker -> %s", done_uri)
        except Exception as e:  # never fail a good stitch because the marker push hiccuped
            logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)


    def cmd_compile(args: argparse.Namespace) -> int:
        """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
stitch is the
        2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
        the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
        the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
there is no
        forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
        bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
        from backend import orchestration
        from backend.config import StitchingConfig

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
        os.makedirs(scratch, exist_ok=True)
        # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
        config.output.output_dir = scratch
        # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
value as a
        # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
        # destination even if the override were dropped.
        if not (config.storage.output_root or "").strip():
            config.storage.output_root = args.out_uri

        spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
        video_id = str(spec.get("video_id") or "")
        out_name = str(spec.get("output_name") or "")
        source_ref = str(spec.get("video_uri") or "")
        locale = spec.get("locale")
        time_pairs = [
            (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
        ]
```

```
496         stitching = StitchingConfig(**(spec.get("stitching") or {}))
497
498         print(
499             f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
500             f"encoder={stitching.encoder} out={out_name}"
501         )
502
503         result = orchestration._stitch_reel(
504             config,
505             stitching,
506             video_id,
507             source_ref,
508             time_pairs,
509             out_name,
510             locale,
511             lambda stage: print(f"[worker] compile: {stage}"),
512         )
513         status = str(result.get("status") or "failed")
514         rc = 0 if status == "success" else 3
515         if status != "success":
516             print(f"[worker] compile FAILED: {result.get('message')}")
517
518         # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
519         if getattr(args, "done_uri", None):
520             _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
521         return rc
522
523
524     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
525     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
526     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
527
528     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
```

35. assistant (2026-07-08T06:09:25.005Z)

Let me read the worker `build_parser`/`main` dispatch and `_apply_overrides` to confirm rc propagation and overrides.

36. user (2026-07-08T06:09:28.323Z)

```
905         )
906         pt.add_argument(
907             "--corpus-uri",
908             required=True,
909             help="prefix containing labels/ (+ videos/ for "
910             "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
911         )
912         pt.add_argument(
913             "--out-uri",
914             required=True,
915             help="output prefix (weights/<version>/ is appended)",
916         )
917         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
918         pt.add_argument(
919             "--trajectory-source",
920             choices=["synth", "detector"],
921             default="synth",
922             help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
923             "detector = REAL shuttle trajectories via the resolved DetectorRunner "
924             "(needs videos/ in the corpus; honours detector_impl).",
```

```
925         )
926         pt.add_argument(
927             "--segmenter",
928             default="wasb_hybrid",
929             help="trajectory hybrid whose detector is used for --trajectory-source
detector",
930         )
931         pt.add_argument(
932             "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
933         )
934         pt.add_argument("--config", default=None)
935         pt.add_argument("--scratch", default=None)
936         pt.add_argument(
937             "--fps",
938             type=float,
939             default=30.0,
940             help="synth trajectory fps (detector probes the video)",
941         )
942         pt.add_argument(
943             "--frame-width",
944             type=float,
945             default=1280.0,
946             help="synth trajectory frame width (detector probes the video)",
947         )
948         pt.add_argument("--set", action="append", default=[], metavar="k=v")
949         pt.set_defaults(func=cmd_train)
950         return p
951
952
953     def _setup_logging(verbose: bool) -> None:
954         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
955         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
956         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
957         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
958         entry), not at import, so importing the module never reconfigures a caller's
logging."""
959         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
960         # configured logging at import), which would leave the level at WARNING and re-
suppress the
961         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
962         logging.basicConfig(
963             level=logging.DEBUG if verbose else logging.INFO,
964             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
965             datefmt="%H:%M:%S",
966             force=True,
967         )
968
969
970     def main(argv=None) -> int:
971         args = build_parser().parse_args(argv)
972         _setup_logging(getattr(args, "verbose", False))
973         return args.func(args)
974
975
976     if __name__ == "__main__":
977         sys.exit(main())
```

37. user (2026-07-08T06:09:34.734Z)

```
61:def _apply_overrides(config: Any, sets: List[str]) -> None:
```

38. user (2026-07-08T06:09:38.115Z)

```
61      def _apply_overrides(config: Any, sets: List[str]) -> None:
62          """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63          for kv in sets or []:
64              key, sep, val = kv.partition("=")
65              if not sep:
66                  raise ValueError(f"--set expects k=v, got {kv!r}")
67              parts = key.split(".")
68              obj = config
69              for p in parts[:-1]:
70                  obj = getattr(obj, p)
71              setattr(obj, parts[-1], _coerce(val))
72
73
74      def _write_intervals_csv(segments: List[dict], path: str) -> None:
75          """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
76          artifact (same schema as the rally-annotator labels)."""
77          with open(path, "w", newline="") as f:
78              w = csv.writer(f)
79              w.writerow(["start", "end", "shot_count", "ending_reason"])
80              for s in segments:
81                  w.writerow(
82                      [
83                          f"{float(s.get('start_time', 0.0)):.3f}",
84                          f"{float(s.get('end_time', 0.0)):.3f}",
85                          s.get("shot_count", ""),
86                          s.get("ending_reason", s.get("shot_count_confidence", "")),
87                      ]
88                  )
89
90
91      def _write_report_json(
92          video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93      ) -> None:
94          with open(path, "w") as f:
95              json.dump(
96                  {
97                      "video_id": video_id,
98                      "status": status,
99                      "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [
105                         {
106                             "start": float(s.get("start_time", 0.0)),
107                             "end": float(s.get("end_time", 0.0)),
108                         }
109                         for s in segments
110                     ],
111                 },
112                 f,
113                 indent=2,
114             )
115
116
117     def _serialize_and_push_outputs(
118         scratch: str,
119         out_uri: str,
```

```
120         video_id: str,
121         segments: List[dict],
122         status: str,
123         storage_cfg: dict,
124         video_uri: str = "",
125     ) -> List[str]:
126         """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127         ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129         Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
```

39. assistant (2026-07-08T06:10:16.470Z)

Let me read the `compile_highlights` region in the compiler to verify the concat stays `-c copy` and encoder is applied uniformly.

40. user (2026-07-08T06:10:19.779Z)

```
495                 )
496
497             # Defense-in-depth: never let the output name escape output_dir (the API also
498             # validates this at the request boundary).
499             output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
500
501             # Probe video duration so we can detect out-of-range segments
502             video_duration = self._get_video_duration(raw_video_path)
503             if video_duration > 0:
504                 logger.info(f"Source video duration: {video_duration:.2f}s")
505             else:
506                 logger.warning(
507                     "Video duration unknown. Cannot validate segment boundaries against
video length."
508                 )
509
510             # We will create temporary clips and stitch them using FFmpeg concat demuxer
511             temp_clips = []
512             skipped_segments = []
513
514             # Create a temp directory within output_dir
515             with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
516                 logger.info(
517                     f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
518                 )
519                 # Build the (optional) watermark filtergraph once; it is applied during
every
520                 # per-clip re-encode so the mark is baked into each clip identically.
521                 watermark_filter = self._watermark_filter(tmp_dir)
522                 if watermark_filter:
523                     logger.info("[watermark] applying branding overlay to each clip.")
524                 # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip
shares one codec
525                 # and the final concat stays -c copy (#521).
526                 video_codec_args, chosen_encoder = self._select_encoder()
527                 if chosen_encoder != "libx264":
528                     logger.info("[compile] encoding clips with %s (hardware).",
chosen_encoder)
529                 for idx, (raw_start, raw_end) in enumerate(segments):
530                     clip_path, skip_record = self._trim_one_segment(
531                         idx,
532                         raw_start,
533                         raw_end,
```

```
                        segments,
                        video_duration,
                        tmp_dir,
                        raw_video_path,
                        watermark_filter,
                        video_codec_args,
                    )
                    if clip_path is not None:
                        temp_clips.append(clip_path)
                    if skip_record is not None:
                        skipped_segments.append(skip_record)

                # Report summary of skipped segments
                if skipped_segments:
                    logger.info(
                        f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:"
                    )
                    for seg_idx, seg_start, seg_end, reason in skipped_segments:
                        logger.info(
                            f"  - Segment #{seg_idx + 1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}"
                        )
                    logger.info("")

                if not temp_clips:
                    raise RuntimeError(
                        f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
                        f"All {len(segments)} segments were skipped. "
                        f"Check the errors above for details. "
                        f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
                    )

                logger.info(
                    f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch..."
                )

                # Create a list file for FFmpeg concat demuxer
                list_file_path = os.path.join(tmp_dir, "clips.txt")
                # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
                # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
                with open(list_file_path, "w", encoding="utf-8") as f:
                    for clip in temp_clips:
                        # The concat demuxer resolves relative entries against the list
file's
                        # directory. Use absolute paths so a temp-dir clip does not become
                        # output/tmp.../output/tmp.../clip_0.mp4 on Windows.
                        safe_path = Path(clip).resolve().as_posix().replace("'", "'\\''")
                        f.write(f"file '{safe_path}'\n")

                logger.info("Stitching clips together...")
                # Stitch them together without re-encoding (since they were encoded with
identical settings)
                concat_cmd = [
                    ffmpeg_bin(),
                    "-y",
                    "-f",
                    "concat",
                    "-safe",
                    "0",
                    "-i",
                    list_file_path,
```

```
592                     "-c",
593                     "copy",
594                     output_path,
595                 ]
596
597                 try:
598                     timeout = ffmpeg_timeout_sec()
599                     subprocess.run(
600                         concat_cmd, capture_output=True, check=True, timeout=timeout
601                     )
602
603                     # Final verification
604                     if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
605                         raise RuntimeError(
606                             f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
607                             f"but output file is missing or empty: {output_path}"
608                         )
609
610                     output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
611                     logger.info(
612                         f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
MB)"
613                     )
614                     return output_path
615                 except subprocess.CalledProcessError as e:
616                     stderr_msg = e.stderr.decode(errors="replace").strip()
617                     raise RuntimeError(
618                         f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
619                         f"  Command: {' '.join(concat_cmd)}\n"
620                         f"  FFmpeg stderr: {stderr_msg[-500:]}"
621                     ) from e
622                 except subprocess.TimeoutExpired as e:
623                     raise RuntimeError(
624                         f"[COMPILER ERROR] FFmpeg concatenation timed out after
{timeout_label(timeout)}.\n"
625                         f"  Command: {' '.join(concat_cmd)}"
```

41. assistant (2026-07-08T06:10:28.723Z)

Let me verify the compiler imports (`Dict`, `Tuple`, `subprocess`, `ffmpeg_timeout_sec`, `ffmpeg_bin`) are present.

42. user (2026-07-08T06:10:31.831Z)

```
1       import logging
2       import os
3       import subprocess
4       import tempfile
5       from pathlib import Path
6       from typing import Any, Dict, List, Optional, Tuple, Union
7
8       from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
9           ffmpeg_bin,
10          ffmpeg_timeout_sec,
11          ffprobe_bin,
12          ffprobe_timeout_sec,
13          timeout_label,
14      )
15
16      logger = logging.getLogger(__name__)
17
18      # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
19      # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
```

```
20      # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
21      # backend/pipeline/stitcher/, so parents[2] == backend/.
22      _BUNDLED_FONT = (
```

43. assistant (2026-07-08T06:11:16.761Z)

Let me review the tests to assess coverage of the failure modes, especially the worker mirror-upload path.

44. user (2026-07-08T06:11:20.839Z)

```
4:(``deployment.phase_placement.compile=cloud_run_l4``), mirroring the DETECT dispatch ? all
GPU-free +
50:            "reel_uri": f"gs://bucket/highlights/{vid}/{out_name}",
108:def test_compile_dispatches_to_l4_and_returns_reel(compile_env):
123:def test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
145:    assert "storage.output_root=" in joined  # bucket forwarded so the worker mirrors the
reel
148:def test_compile_dispatch_forwards_stitch_config_overrides(compile_env):
150:    (storage.output_root/backend) ? else the worker can't read the spec or mirror the
reel."""
160:def test_compile_cloud_failure_returns_failed(compile_env):
173:def test_compile_default_stitches_in_process_no_dispatch(compile_env, monkeypatch):
197:def test_compile_missing_video_fails_before_dispatch(compile_env):
```

45. user (2026-07-08T06:11:25.316Z)

```
1       """Compile/stitch ? L4 worker dispatch (#521).
2
3       Proves the control-plane can OFFLOAD the reel compile/stitch to the Cloud Run L4 worker
by config
4       (``deployment.phase_placement.compile=cloud_run_l4``), mirroring the DETECT dispatch ?
all GPU-free +
5       cloud-free via a FAKE ``CloudRunJobClient`` that simulates the compile worker (reads the
staged
6       compile-spec + writes the reel + done-marker into a local "bucket"). Cardinal invariant:
DEFAULT
7       (compile on the control-plane) still stitches in-process, byte-identically.
8       """
9
10      import json
11      import os
12
13      import pytest
14
15      import backend.compute as compute_pkg
16      import backend.orchestration as orch
17      from backend.compute.cloud_run import CloudRunJobClient
18      from backend.config import AppConfig
19      from backend.infrastructure.database import Database
20      from backend.infrastructure.execution_policy import ExecutionPolicy
21
22      _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
23
24
25      class _FakeCompileClient(CloudRunJobClient):
26          """Simulates the Cloud Run compile worker: on dispatch, READ the staged compile-spec
(proving the
27          control-plane resolved + staged it), then write the reel done-marker into the local
bucket exactly
28          where the real worker would, so the dispatcher's bucket-poll resolves."""
29
30          def __init__(self, *, returncode=0, video_id=None):
```

```
            self.calls: list[list[str]] = []
            self.spec: dict | None = None
            self.returncode = returncode
            self._video_id = video_id

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append(list(argv))
            spec_uri = argv[argv.index("--spec-uri") + 1]
            done_uri = argv[argv.index("--done-uri") + 1]
            with open(spec_uri, encoding="utf-8") as f:
                self.spec = json.load(f)
            vid = self._video_id or self.spec["video_id"]
            out_name = self.spec["output_name"]
            status = "success" if self.returncode == 0 else "failed"
            marker = {
                "video_id": vid,
                "returncode": self.returncode,
                "status": status,
                "download_url": f"/api/highlights/{vid}/{out_name}",
                "reel_uri": f"gs://bucket/highlights/{vid}/{out_name}",
                "message": "" if self.returncode == 0 else "stitch blew up",
            }
            os.makedirs(os.path.dirname(done_uri), exist_ok=True)
            with open(done_uri, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            return "exec-compile-1"

        def cancel_execution(self, execution):  # pragma: no cover - happy path never
cancels
            raise AssertionError("cancel must never fire on the happy path")


    def _inject_fake(monkeypatch, fake):
        real = compute_pkg.get_compute_backend
        monkeypatch.setattr(
            compute_pkg,
            "get_compute_backend",
            lambda config, **kw: real(config, run_client=fake, policy=_FAST, token="tok",
**kw),
        )


    @pytest.fixture
    def compile_env(tmp_path, monkeypatch):
        """A control-plane DB with a video + 3 play segments, wired for cloud with compile
placed on the
        L4 and a LOCAL bucket (so the fake worker's spec/marker are plain file
reads/writes)."""
        db = Database(str(tmp_path / "t.db"))
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        video_id = "vidCompile"
        src = tmp_path / "source.mp4"
        src.write_bytes(b"SRC")
        db.add_video(video_id, str(src), "processed", [])
        sid1 = db.add_play_segment(video_id, 2.0, 6.0, metadata={"shot_count": 5})
        sid2 = db.add_play_segment(video_id, 10.0, 14.0, metadata={"shot_count": 3})
        sid3 = db.add_play_segment(video_id, 20.0, 22.0, metadata={"shot_count": 4})
        cfg = AppConfig.model_validate(
            {
                "deployment": {
                    "compute_target": "cloud_run",
                    "phase_placement": {"compile": "cloud_run_l4"},
                },
                "storage": {"backend": "local", "output_root": str(bucket)},
```

```
            }
        )
        return db, cfg, video_id, [sid1, sid2, sid3], bucket, monkeypatch


    def _job(video_id, segment_ids, name="reel.mp4", locale=None):
        return {
            "id": "cj1",
            "video_id": video_id,
            "kind": "compile",
            "params": json.dumps(
                {"segment_ids": segment_ids, "output_filename": name, "locale": locale}
            ),
        }


    def test_compile_dispatches_to_l4_and_returns_reel(compile_env):
        db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
        fake = _FakeCompileClient()
        _inject_fake(monkeypatch, fake)

        out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "myreel.mp4"))

        assert out["status"] == "success"
        assert out["video_id"] == video_id
        assert out["download_url"] == f"/api/highlights/{video_id}/myreel.mp4"
        assert out["compute"]["path"] == "cloud_run"
        assert out["compute"]["execution"] == "exec-compile-1"
        assert len(fake.calls) == 1  # exactly one Cloud Run Job execution


    def test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
        """The staged compile-spec carries the RESOLVED time-pairs (padding-agnostic
start/end from the DB)
        and the L4 encoder (deployment.cloud_stitch_encoder = h264_nvenc by default) ? so
the worker
        hardware-encodes and needs no control-plane DB."""
        db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
        fake = _FakeCompileClient()
        _inject_fake(monkeypatch, fake)

        orch._execute_compile(cfg, db, _job(video_id, seg_ids))

        assert fake.spec is not None
        assert fake.spec["video_id"] == video_id
        assert fake.spec["video_uri"].endswith("source.mp4")
        # 3 resolved (start, end) pairs from the DB rows
        assert fake.spec["time_pairs"] == [[2.0, 6.0], [10.0, 14.0], [20.0, 22.0]]
        # the L4 stitch uses NVENC (forwarded from deployment.cloud_stitch_encoder)
        assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
        # the argv is a `compile` subcommand carrying the spec + done marker
        argv = fake.calls[0]
        assert argv[0] == "compile"
        assert "--spec-uri" in argv and "--done-uri" in argv
        joined = " ".join(argv)
        assert "storage.output_root=" in joined  # bucket forwarded so the worker mirrors
the reel


    def test_compile_dispatch_forwards_stitch_config_overrides(compile_env):
        """The worker Job has no DEPLOYMENT_PROFILE, so the dispatch forwards the bucket
coordinates
        (storage.output_root/backend) ? else the worker can't read the spec or mirror the
reel."""
        db, cfg, video_id, seg_ids, bucket, monkeypatch = compile_env
```

```
152         fake = _FakeCompileClient()
153         _inject_fake(monkeypatch, fake)
154         orch._execute_compile(cfg, db, _job(video_id, seg_ids))
155         joined = " ".join(fake.calls[0])
156         assert f"storage.output_root={bucket}" in joined
157         assert "storage.backend=local" in joined
158 
159 
160     def test_compile_cloud_failure_returns_failed(compile_env):
161         """A worker stitch failure (rc != 0) surfaces as a failed job with the worker's
message ? the SPA
162         gets a clean verdict, not a crash."""
163         db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
164         fake = _FakeCompileClient(returncode=3)
165         _inject_fake(monkeypatch, fake)
166 
167         out = orch._execute_compile(cfg, db, _job(video_id, seg_ids))
168         assert out["status"] == "failed"
169         assert "stitch blew up" in out["message"]
170         assert out["video_id"] == video_id
171 
172 
173     def test_compile_default_stitches_in_process_no_dispatch(compile_env, monkeypatch):
174         """DEFAULT (compile on the control-plane): _execute_compile stitches in-process via
_stitch_reel
175         and NEVER dispatches to Cloud Run. The stitch core is stubbed so no real ffmpeg
runs."""
176         db, cfg, video_id, seg_ids, _bucket, _mp = compile_env
177         # flip compile back to the control-plane (the default) ? everything else stays
cloud-wired
178         cfg.deployment.phase_placement.compile = "control_plane"
179         fake = _FakeCompileClient()
180         _inject_fake(monkeypatch, fake)
181         seen = {}
182 
183         def _fake_stitch(c, stitching, vid, source, pairs, name, locale, notify):
184             seen["pairs"] = pairs
185             seen["source"] = source
186             return {"status": "success", "video_id": vid, "download_url":
f"/api/highlights/{vid}/{name}"}
187 
188         monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
189 
190         out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "local.mp4"))
191         assert out["status"] == "success"
192         assert fake.calls == []  # NEVER dispatched ? the control-plane stitched in-process
193         assert seen["pairs"] == [(2.0, 6.0), (10.0, 14.0), (20.0, 22.0)]
194         assert seen["source"].endswith("source.mp4")
195 
196 
197     def test_compile_missing_video_fails_before_dispatch(compile_env):
198         """A compile for an unknown video_id fails at resolve (404) before any dispatch."""
199         db, cfg, _video_id, _seg_ids, _bucket, monkeypatch = compile_env
200         fake = _FakeCompileClient()
201         _inject_fake(monkeypatch, fake)
202         out = orch._execute_compile(cfg, db, _job("nope", [1, 2]))
203         assert out["status"] == "failed"
204         assert fake.calls == []  # never dispatched ? resolve failed first
205 
```

46. assistant (2026-07-08T06:11:53.085Z)

Let me check the worker `cmd_compile` tests for coverage of the marker and mirror behavior.

47. user (2026-07-08T06:11:57.498Z)

```
194:    assert rc == 0
268:    assert rc == 0  # still a clean run, just empty ? but now EXPLAINED
279:def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
283:    as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
314:    assert rc == 3
319:    # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
320:    marker = json.loads(done.read_text())
321:    assert marker["returncode"] == 3 and marker["status"] == "failed"
322:    assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
327:    through cmd_infer -> _fail(3) -> a status="failed" report.json + done-marker. Locks the
two
328:    halves (segmenter Failure + worker report/marker) TOGETHER, not just via the _FailSeg
stub."""
374:    assert rc == 3
377:    marker = json.loads(done.read_text())
378:    assert marker["returncode"] == 3 and marker["status"] == "failed"
410:    assert rc == 2
412:    # success): report.json status="failed" + the done-marker rc 2 / status="failed".
415:    marker = json.loads(done.read_text())
416:    assert marker["returncode"] == 2 and marker["status"] == "failed"
451:    assert rc == 0  # a clip that WOULD fail re-validation still succeeds ? the gate is
trusted
546:    assert rc == 0  # before the fix: every clip skipped -> rc 2 ("<2 videos")
614:        rc == 0
702:    assert rc == 0
755:    assert rc == 2  # only 1 of 2 labels joined -> below the leave-one-video-out floor
802:    assert rc == 0
891:    assert rc == 0
938:        rc == 2
952:def test_parser_compile_accepts_spec_and_done_uris():
968:    assert args.done_uri == "gs://b/_compile/tok.done"
989:def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
990:    """cmd_compile reads the staged spec, runs the SHARED _stitch_reel core (VideoCompiler
stubbed so
991:    no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's
bucket-poll."""
1012:            "reel_uri": f"gs://b/highlights/{video_id}/{out_name}",
1015:    monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
1038:    assert rc == 0
1043:    marker = json.loads(open(done, encoding="utf-8").read())
1044:    assert marker["returncode"] == 0 and marker["status"] == "success"
1045:    assert marker["download_url"] == "/api/highlights/vidReel/reel.mp4"
1048:def test_cmd_compile_failure_returns_rc3_and_failed_marker(tmp_path, monkeypatch):
1049:    """A stitch failure surfaces as rc 3 + a failed marker (rc != 0) so the dispatcher's
poll
1062:        "_stitch_reel",
1084:    assert rc == 3
1085:    marker = json.loads(open(done, encoding="utf-8").read())
1086:    assert marker["returncode"] == 3 and marker["status"] == "failed"
1089:def test_cmd_compile_no_done_uri_writes_no_marker(tmp_path, monkeypatch):
1090:    """DEFAULT-OFF: a direct-CLI stitch with no --done-uri writes no marker."""
1100:        orch, "_stitch_reel", lambda *a, **k: {"status": "success", "video_id": "vidReel"}
1119:    assert rc == 0
```

48. assistant (2026-07-08T06:12:03.013Z)

Let me confirm the `/api/highlights` fallback behavior to solidify the worker mirror-failure impact.

49. user (2026-07-08T06:12:07.995Z)

```
37:    FileResponse,
246:    signed_reel_url,
```

```
513:    Mirrors /api/highlights' safety (bare-name + containment checks)."""
527:    return FileResponse(path, media_type=media, filename=name)
626:    # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
635:        return FileResponse(proxy_path, media_type="video/mp4")
671:    return FileResponse(local, media_type=media)
1041:@app.get("/api/highlights/{video_id}/{filename}")
1042:def download_highlight(video_id: str, filename: str, request: Request):
1054:    # exists/sign logic (with the active storage config) lives in
orchestration.signed_reel_url to keep
1057:        signed = signed_reel_url(request.app.state.config, video_id, name)
1061:        logger.warning("[highlights] signed-url path failed for %s; serving local: %s",
video_id, e)
1079:    return FileResponse(path, media_type=media, filename=name)
1099:def compile_highlights(req: CompileRequest, request: Request):
1279:        help="Directory where processed clips/highlights are saved (overrides config.json
output.output_dir)",
1388:            res_path = compiler.compile_highlights(
```

50. user (2026-07-08T06:12:11.464Z)

```
1041    @app.get("/api/highlights/{video_id}/{filename}")
1042    def download_highlight(video_id: str, filename: str, request: Request):
1043        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
1044        (which only writes into output_dir; the browser previously got back an unreachable
1045        server-local path)."""
1046        try:
1047            name = safe_output_filename(filename)
1048        except ValueError as e:
1049            raise HTTPException(status_code=400, detail=str(e)) from e
1050        if not request.app.state.db.get_video(video_id):
1051            raise HTTPException(status_code=404, detail="Indexed video record not found.")
1052        # #463: when the reel was mirrored to the output bucket, 307 to a short-lived signed
URL (bytes flow
1053        # client<->store, bypassing Cloud Run's ~32 MiB response cap + the ephemeral local
/tmp). The
1054        # exists/sign logic (with the active storage config) lives in
orchestration.signed_reel_url to keep
1055        # this module lean; best-effort ? any error falls back to the local file below.
1056        try:
1057            signed = signed_reel_url(request.app.state.config, video_id, name)
1058            if signed:
1059                return RedirectResponse(signed, status_code=307)
1060        except Exception as e:  # noqa: BLE001 ? signing/exists hiccup ? serve the local
file instead
1061            logger.warning("[highlights] signed-url path failed for %s; serving local: %s",
video_id, e)
1062        output_dir = (
1063            getattr(
1064                getattr(request.app.state.config, "output", None), "output_dir", "output"
1065            )
1066            or "output"
1067        )
1068        path = os.path.join(output_dir, name)
1069        try:
1070            path = resolve_under_root(path, output_dir)
1071        except ValueError as e:
1072            raise HTTPException(status_code=400, detail=str(e)) from e
1073        if not os.path.isfile(path):
1074            raise HTTPException(
1075                status_code=404,
1076                detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
1077            )
1078        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
1079        return FileResponse(path, media_type=media, filename=name)
```

```
1080
```

51. assistant (2026-07-08T06:13:26.347Z)

```
Let me verify how `storage.fetch`/`put` route (by URI scheme vs config backend), which
determines whether the compile spec round-trip is robust to a missing `storage.backend`
override.
```

52. user (2026-07-08T06:13:31.415Z)

```
66:    return parse_uri(uri)
149:def get_storage(
170:def fetch(
179:    ref = parse_uri(uri)
185:def put(
193:    ref = parse_uri(uri)
200:def exists(uri: str, *, config: Optional[Dict[str, Any]] = None) -> bool:
202:    ref = parse_uri(uri)
220:    ref = parse_uri(uri)
228:    ref = parse_uri(prefix_uri)
242:    ref = parse_uri(uri)
```

53. user (2026-07-08T06:13:38.819Z)

```
149     def get_storage(
150         backend: Optional[str] = None,
151         *,
152         config: Optional[Dict[str, Any]] = None,
153         **overrides,
154     ) -> StorageBackend:
155         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
156         the implementation; its per-backend settings come from ``config[backend]`` merged
with
157         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
158         cfg = config or {}
159         name = backend or cfg.get("backend", "local")
160         # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
161         # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
162         # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
163         settings: Dict[str, Any] = dict(
164             cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
165         )
166         settings.update(overrides)
167         return _backend_class(name)(**settings)
168
169
170     def fetch(
171         uri: str,
172         dst: "str | os.PathLike[str] | None" = None,
173         *,
174         config: Optional[Dict[str, Any]] = None,
175         overwrite: bool = False,
176     ) -> str:
177         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
178         (no copy); for s3/gcs it downloads to ``dst``."""
179         ref = parse_uri(uri)
180         return get_storage(ref.backend, config=config).fetch_to_local(
181             ref, dst, overwrite=overwrite
182         )
```

```
183
184
185     def put(
186         local_path: "str | os.PathLike[str]",
187         uri: str,
188         *,
189         config: Optional[Dict[str, Any]] = None,
190         content_type: Optional[str] = None,
191     ) -> str:
192         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
193         ref = parse_uri(uri)
194         get_storage(ref.backend, config=config).put(
195             local_path, ref, content_type=content_type
196         )
197         return uri
198
199
200     def exists(uri: str, *, config: Optional[Dict[str, Any]] = None) -> bool:
201         """True if the object at ``uri`` exists (a bucket object or a local file)."""
202         ref = parse_uri(uri)
203         return bool(get_storage(ref.backend, config=config).exists(ref))
```

54. assistant (2026-07-08T06:15:54.553Z)

I have traced the full flow. Let me do a final verification of the exact line numbers of the mirror-upload block in the post-change `orchestration.py`.

55. user (2026-07-08T06:15:57.876Z)

```
1251         # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1252         # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1253         # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1254         # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
1255         # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
1256         reel_uri = reel_object_uri(cfg, video_id, out_name)
1257         if reel_uri:
1258             try:
1259                 notify("uploading reel")
1260                 # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1261                 # project/client) is used, not a fresh default.
1262                 storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1263             except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1264                 logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1265
1266         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1267         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1268         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1269         notify("finished")
1270         return {
1271             "status": "success",
1272             "filepath": out_path,
1273             "download_url": download_url,
1274             "video_id": video_id,
1275             "reel_uri": reel_uri or "",
1276         }
1277
```

56. assistant (2026-07-08T06:16:18.810Z)

I've completed my correctness trace of the #521 dispatch/placement/worker-compile/spec round-trip logic. Let me report my findings.

Summary of what I verified as CORRECT (no defect):
- Validation delegation is airtight: `_will_detect_on_cloud` and `_maybe_dispatch_remote` gate identically (both on `resolve_phase_placement(SEGMENT)` + `_cloud_available`); `get_compute_backend(target_override="cloud_run")` never returns None when `_cloud_available` is true, so there is no "nobody validates" gap. Config guard matches the resolver's legacy default.
- DEFAULT-OFF byte-identical: unset placement ? control_plane/in-process; encoder default libx264 reproduces the old inline `-c:v libx264 -preset superfast -crf 22`.
- Marker prefixes distinct (`_compile/` vs `_dispatch/`); rc/marker misread paths default to failure; crash/timeout ? clean failed-job via shared `_await_marker`.
- Encoder is resolved once (all-or-nothing), concat stays `-c copy`; storage routes by URI scheme so the missing-`storage.backend` override isn't a routing bug.
- time-pairs / ordering / min_shot_count filter applied once on the control-plane, consistent between local and cloud paths; download_url identical across both.

The one real defect:

57. user (2026-07-08T06:16:29.856Z)

Structured output provided successfully